BUSN 5000 Project Guide

Chris Cornwell and Abbi Cormier

May 18, 2026

Table of contents

Preface **1**
How to use this guide . . . 1
Acknowledgments . . . 1

1 Setup **3**
1.1 Create your BUSN 5000 folder . . . 3
1.2 Install R and RStudio . . . 3
1.2.1 Windows . . . 3
1.2.2 macOS . . . 4
1.3 Install the required R packages . . . 5
1.4 Create the subfolders inside `Project/` . . . 5
1.5 Sort the Pre-project file pack into the right places . . . 5
1.6 Create the `.Rproj` file . . . 6
1.7 Run the setup check . . . 7
1.8 Common setup mistakes . . . 7
1.8.1 Mistake 1: `project_slidedeck.css` in the wrong folder . . . 7
1.8.2 Mistake 2: Working from your Downloads folder . . . 7
1.8.3 Mistake 3: Knitting before running the R script . . . 7
1.9 What happens next . . . 8

2 Pre-project **9**
2.1 What you'll produce . . . 9
2.2 Workflow . . . 9
2.2.1 Step 1: Open the project in RStudio . . . 9
2.2.2 Step 2: Run `LF.R` . . . 9
2.2.3 Step 3: Open `pre_project.Rmd` . . . 10
2.2.4 Step 4: Fill in the YAML author line . . . 10
2.2.5 Step 5: Complete the code chunk on Slide 2 . . . 11
2.2.6 Step 6: Fill in the paragraph on Slide 3 . . . 11
2.2.7 Step 7: Knit the Rmd to HTML . . . 12
2.2.8 Step 8: Compare your output to the reference . . . 12
2.2.9 Step 9: Save the HTML as PDF . . . 12
2.2.10 Step 10: Submit to Gradescope . . . 12
2.3 Common pre-project mistakes . . . 12
2.4 What happens next . . . 13

3 Main Project **15**
3.1 Project-course alignment . . . 15
3.2 Before you start . . . 16
3.3 General rules . . . 16
3.3.1 Slide format . . . 16
3.3.2 Write-up line limits . . . 16
3.3.3 Echo settings . . . 17
3.3.4 HTML commands wrapping chunks . . . 17

3.3.5 The YAML block . . . 17
3.3.6 The setup chunk . . . 18
3.4 A note on units . . . 18
3.5 Slide-by-slide instructions . . . 18
3.5.1 Front matter (slides 1–2) . . . 18
3.5.2 Introduction (slides 3–4) . . . 19
3.5.3 Data (slides 5–8) . . . 19
3.5.4 Baseline earnings distributions (slides 9–12) . . . 20
3.5.5 The career gender gap (slides 13–20) . . . 21
3.5.6 Explaining the gender wage gap (slides 21–30) . . . 22
3.5.7 Conclusion (slides 31–32) . . . 24
3.5.8 Appendix (slides 33–35) . . . 24
3.6 Submission . . . 25
3.7 Common pitfalls . . . 25
3.8 What happens next . . . 25

4 Progress Check **27**
4.1 Why bother . . . 27
4.2 What you'll do . . . 27
4.3 What's graded . . . 27
4.4 Rubric . . . 28
4.5 What's *not* graded . . . 28
4.6 Submission . . . 29
4.7 Common pitfalls . . . 29
4.8 What happens next . . . 29

5 Submission **31**
5.1 Filename conventions . . . 31
5.2 Late submission policy . . . 31
5.3 The submission workflow at a glance . . . 31
5.4 Saving HTML as PDF . . . 32
5.4.1 Google Chrome (and Microsoft Edge) . . . 32
5.4.2 Firefox . . . 33
5.4.3 Safari . . . 35
5.5 Pre-project reference: what your three slides should look like . . . 36
5.5.1 Reference slide 1 — Title slide . . . 37
5.5.2 Reference slide 2 — Read `LF.csv` and print its contents . . . 38
5.5.3 Reference slide 3 — Document the contents of `LF.csv` . . . 39
5.6 Final Project reference: what your 35 slides should look like . . . 39
5.6.1 Front matter (slides 1–2) . . . 40
5.6.2 Introduction (slides 3–4) . . . 42
5.6.3 Data (slides 5–8) . . . 44
5.6.4 Baseline earnings distributions (slides 9–12) . . . 48
5.6.5 The career gender gap (slides 13–20) . . . 52
5.6.6 Explaining the gender wage gap (slides 21–30) . . . 60
5.6.7 Conclusion (slides 31–32) . . . 70
5.6.8 Appendix (slides 33–35) . . . 72
5.7 If your output doesn't match the reference . . . 74

6 Common Errors **75**
6.1 How to use this chapter . . . 75
6.2 Setup errors . . . 75
6.2.1 CSS not loading . . . 75
6.2.2 Working from your Downloads folder . . . 75
6.2.3 Required files missing or in the wrong subfolder . . . 75
6.3 Workflow errors . . . 76
6.3.1 Knitting before running the R script . . . 76
6.3.2 Variable name confusion . . . 76
6.3.3 Touching the setup chunk . . . 76
6.4 Rendering errors (the knit step) . . . 76
6.4.1 Leaving `eval = FALSE` on a completed chunk . . . 76
6.4.2 Editing the YAML beyond the author line . . . 77
6.4.3 Using the Knit dropdown instead of the Knit button . . . 77
6.5 Output format errors . . . 77
6.5.1 Wrong CSS . . . 77
6.5.2 Wrong CSS + wrong orientation . . . 78
6.5.3 Wrong orientation . . . 79
6.5.4 Long landscape . . . 80
6.5.5 Portrait . . . 81
6.5.6 Portrait + wrong CSS . . . 82
6.5.7 LaTeX slides (knit-to-PDF, slide format) . . . 83
6.5.8 LaTeX document (knit-to-PDF, document format) . . . 84
6.5.9 Not knit . . . 85
6.5.10 Wrong file . . . 86
6.6 Content errors . . . 87
6.6.1 Numbers in your paragraph don't match `LF.csv` (pre-project) . . . 87
6.6.2 Wrong filename . . . 87
6.7 Getting help . . . 88

7 R and AI Resources **89**
7.1 Resources for learning and using R . . . 89
7.1.1 IDEs for R . . . 89
7.1.2 Learning R . . . 89
7.1.3 Workflow . . . 89
7.2 AI resources . . . 90
7.2.1 Working with AI . . . 90
7.2.2 AI for coding . . . 90
7.2.3 Learning more about AI . . . 90

Preface

Welcome to the **BUSN 5000 Project Guide** — the canonical reference for the Pre-project, Project Progress Check, and Final Project for BUSN 5000 (Intro to Data Science for Business and Economics) at the University of Georgia.

This guide covers:

- How to set up your computer and your project folder
- What the pre-project is and how to complete it
- What the main project is and how to complete it
- What the progress check is and how it's graded
- How to submit your work
- How to recognize and recover from common errors

The guide is the single source of truth. Announcements on eLC remind you of upcoming deadlines and point back here for the rules.

How to use this guide

Read the chapters in order the first time through. After that, treat it as a reference — jump to whatever section you need. Each chapter has its own table of contents in the sidebar.

If you find a section that's wrong, unclear, or out of date, please flag it via the "report an issue" link at the top of any page.

Acknowledgments

The course materials in this guide are by **Chris Cornwell** and **Abbi Cormier**, with substantial contributions from the BUSN 5000 teaching team across many semesters.

1Setup

This chapter walks you through getting your computer ready to do BUSN 5000 project work. By the end of it, you should have:

- R and RStudio installed
- All required R packages installed
- A `Project/` folder on your computer with the correct subfolder structure
- Every file from the Pre-project file pack sorted into the right place
- An `.Rproj` file that opens RStudio into the project
- A green report from the setup-check script

Plan to spend 30–45 minutes the first time through.

> **!** Don't skip this
>
> Almost every "my project won't knit" email we get traces back to something here. If setup is right, the rest of the project is much easier. If setup is wrong, every other step compounds the problem.

1.1 Create your BUSN 5000 folder

Before you do anything project-specific, set up a top-level **`BUSN 5000`** folder on your computer. Put it somewhere you'll remember — your `Documents` folder is a fine default.

Inside `BUSN 5000`, create two subfolders:

```
BUSN 5000/
  Homework/
  Project/
```

- `Homework/` is where you'll save homework assignments and other course materials over the semester.
- `Project/` is the self-contained working directory for the Pre-project, Project Progress Check, and Final Project.

The rest of this chapter is about setting up `Project/`. We won't touch `Homework/` here.

1.2 Install R and RStudio

If you haven't already, install both. Follow the instructions for your operating system. Install R before RStudio.

1.2.1 Windows

Step 1: Install R

1. Go to the CRAN R Project website.
2. Click on "Download R for Windows".
3. Click on "base" and then select the link labeled "Download R x.x.x for Windows" (where x.x.x is the latest version).
4. Open the downloaded `.exe` file and follow the installation prompts, accepting the defaults.

Step 2: Install RStudio

1. Visit the RStudio download page.
2. Click the link to download the Windows installer.
3. Open the downloaded `.exe` file and complete the installation using default settings.

Step 3: Verify the installation

1. Launch RStudio from the Start menu or desktop shortcut.
2. In the Console pane, type `print("Hello, world!")` and press Enter.
3. If you see `[1] "Hello, world!"`, your installation is successful.

1.2.2 macOS

First, identify your Mac's chip. Newer Macs use Apple Silicon (M1, M2, M3, M4, etc.); older ones use Intel processors. You need to know which you have to pick the right R installer.

1. Click the Apple menu (top-left corner) → **About This Mac**.
2. Look at the **Chip** (Apple Silicon) or **Processor** (Intel) line:
   - If it says "Apple M1", "M2", "M3", "M4", or any Apple-labeled chip, you have **Apple Silicon**. You want the `arm64` installer.
   - If it says "Intel Core …", you have an **Intel Mac**. You want the `x86_64` installer.

Step 1: Install R

1. Visit the CRAN R Project website.
2. Click on "Download R for macOS".
3. Select the latest version matching your chip — either `R-x.x.x-arm64.pkg` (Apple Silicon) or `R-x.x.x-x86_64.pkg` (Intel).
4. Open the downloaded `.pkg` file and follow the installation prompts.

Step 2: Install RStudio

1. Go to the RStudio download page.
2. Click the link to download the macOS version of RStudio.
3. Open the downloaded `.dmg` file and drag the RStudio icon to your `Applications` folder.

Step 3: Verify the installation

1. Open RStudio from your `Applications` folder.
2. In the Console pane, type `print("Hello, world!")` and press Return.
3. If you see `[1] "Hello, world!"`, your installation is successful.

1.3 Install the required R packages

Install all seven packages using the **Packages** pane in RStudio:

1. Select the **Packages** tab in the bottom-right pane.
2. Click **Install** at the top of that pane.
3. In the "Install Packages" box, enter the package names separated by spaces or commas:

   ```
   here tidyverse modelsummary ggpmisc car ggrepel kableExtra
   ```

4. Make sure **Install dependencies** is checked.
5. Click **Install**.

Installation takes a few minutes the first time — let it finish completely.

> **Tip**
>
> You only install packages **once per machine**. After that, the templates load them with `library()` calls inside the Rmd setup chunks, which run every time you knit. Don't put `install.packages()` calls inside an `.Rmd` — that would re-install every time you knit.

1.4 Create the subfolders inside `Project/`

Inside your `BUSN 5000/Project/` folder, create these four subfolders:

- `data/`
- `out/`
- `r/`
- `css/`

So your skeleton looks like:

```
BUSN 5000/
  Homework/
  Project/
      data/
      out/
      r/
      css/
```

You'll put files into these subfolders in the next step. The `.Rmd` files themselves will live in the `Project/` root, **not** in any of the subfolders.

1.5 Sort the Pre-project file pack into the right places

When you download the Pre-project file pack from eLC, you'll get a flat bucket of 7 files. Your job is to figure out where each one belongs in your `Project/` directory. This isn't busywork — knowing

which file goes where is itself part of what we want you to learn.

Here's the destination for each file in the pack:

File	Goes in
`pre_project.Rmd`	`Project/` (root)
`preproject_instructions.pdf`	`Project/` (root)
`LF.R`	`Project/r/`
`project_slidedeck.css`	`Project/css/`
`pppub24.csv`	`Project/data/`
`hhpub24.csv`	`Project/data/`
`check_setup_preproject.R`	`Project/r/`

After sorting, your folder should look like:

```
Project/
   pre_project.Rmd
   preproject_instructions.pdf
   data/
       pppub24.csv
       hhpub24.csv
   out/
    (empty for now - LF.R will write LF.csv here)
   r/
       LF.R
       check_setup_preproject.R
   css/
        project_slidedeck.css
```

1.6 Create the `.Rproj` file

The `.Rproj` file is what tells RStudio "this folder is an R project." It anchors all the relative paths the templates use.

To create one:

1. Open RStudio.
2. **File → New Project**.
3. Choose **Existing Directory**.
4. Browse to your `Project/` folder. Click **Create Project**.
5. RStudio re-opens with `Project` loaded as the active project.

You'll see `Project.Rproj` appear in the `Project/` root. From now on, **always open RStudio by double-clicking that `.Rproj` file**. That guarantees your working directory and paths are correct.

1.7 Run the setup check

1. In RStudio's **Files** pane (bottom-right), open the `r/` subfolder.
2. Click **`check_setup_preproject.R`** to open it in the editor.
3. In the editor, **Select All** (Ctrl+A on Windows, Cmd+A on Mac).
4. Click the **Run** button in the top-right of the editor pane (or press Ctrl+Enter / Cmd+Enter).

The script's output appears in the Console pane. You'll see one line per file: either (good) or with a specific instruction telling you what to fix.

> **Note**
>
> If you see any , fix the listed problems and re-run the script. Don't move past Setup until every line is a .

1.8 Common setup mistakes

In our experience, these are the three setup problems we see most often. Each one is preventable with a few seconds of attention.

1.8.1 Mistake 1: `project_slidedeck.css` in the wrong folder

The CSS file styles your slide deck. If it's anywhere but `Project/css/project_slidedeck.css`, your knitted slides will lose all of their formatting and the submission will be rejected. The setup-check script will catch this, but it's a simple thing to get right on the first pass.

1.8.2 Mistake 2: Working from your Downloads folder

If you double-click `pre_project.Rmd` directly from your Downloads folder, RStudio's working directory is Downloads, not your `Project/` folder. The `here()` paths in the `.Rmd` will resolve to the wrong place and you'll get cryptic "file not found" errors.

Always move the files into your `Project/` folder before opening them. Your Downloads folder is a transit area, not a workspace.

1.8.3 Mistake 3: Knitting before running the R script

The `.Rmd` reads output that the `LF.R` script (and later `cpsmar_e.R`) produces. If you haven't run the script yet, the output file doesn't exist, and the `.Rmd` errors out the moment it tries to read `out/LF.csv`.

Workflow order for each stage:

1. Open the `.Rproj`.
2. Run the relevant R script (`LF.R` for pre-project, `cpsmar_e.R` for main project).
3. *Then* open the `.Rmd` and knit it.

The setup-check script can confirm files exist, but only running the script confirms it actually produced the output the `.Rmd` needs.

1.9 What happens next

You're set up. From here:

- For the **Pre-project**, continue to Chapter 3.
- For the **Main Project** (later in the semester), you'll add more files to the same `Project/` folder — see Chapter 4.
- For the **Project Progress Check**, you continue working on the same main project files — see Chapter 5.
- For supplementary materials on R, RStudio, AI tools, and coding practices, see R and AI Resources.

Your `Project/` directory carries through the entire semester. You won't need to re-do Setup unless you're working from a new machine.

2Pre-project

The Pre-project Exercise is a short, required exercise designed to get you and your computer ready for the Main Project. By completing it correctly, you'll have:

- Installed the required R packages
- Set up the proper directory structure on your computer
- Learned how to knit your `.Rmd` to an HTML slide deck and save it in PDF format
- Earned the **5% of your Project grade** that the Pre-project is worth

> **!** All-or-nothing
>
> The Pre-project is worth **5% of your Project grade**, graded all-or-nothing. If your content is correct **and** your slide deck is 100% compliant with the formatting in `pre_project_knitted_template.html`, you earn the full 5%. Any deviation — content error, formatting error, wrong filename, anything — and you earn 0.
> There is no partial credit on the Pre-project.

This chapter assumes you've completed Setup. If you haven't created your `Project/` folder, sorted the file pack, created the `.Rproj`, and run `check_setup_preproject.R`, go do that first.

2.1 What you'll produce

A 3-slide PDF slide deck:

1. **Title slide** — auto-generated from the YAML block; just requires your name.
2. **Read LF.csv and print its contents** — one code chunk you complete.
3. **Document the contents of LF.csv** — one paragraph with six blanks to fill in.

That's it. Three slides.

2.2 Workflow

The order matters. Follow these steps in sequence.

2.2.1 Step 1: Open the project in RStudio

Double-click `Project.Rproj` in your `BUSN 5000/Project/` folder. RStudio opens with the project loaded. You should see "Project" in the top-right corner of the RStudio window.

2.2.2 Step 2: Run `LF.R`

`LF.R` reads the raw CPS data file (`pppub24.csv`), computes the labor force participation rate and its components, and writes the results to `out/LF.csv`. You need to run it before you can knit the Rmd.

1. In RStudio's **Files** pane (bottom-right), open the `r/` subfolder.
2. Click `LF.R` to open it in the editor.
3. **Select All** (Ctrl+A on Windows, Cmd+A on Mac).
4. Click the **Run** button (or press Ctrl+Enter / Cmd+Enter).

The console will print a message like `March 2024 LFPR = 62.7...`. If you see it, the script ran. A new file `LF.csv` now lives in your `out/` subfolder.

> **!** Important
>
> You must run `LF.R` **before** you knit `pre_project.Rmd`. The Rmd reads `out/LF.csv`; if that file doesn't exist, the Rmd errors out.

2.2.3 Step 3: Open `pre_project.Rmd`

In the Files pane, click `pre_project.Rmd` (it lives in the `Project/` root, not in any subfolder). It opens in the editor.

2.2.4 Step 4: Fill in the YAML author line

Near the top of the file, find the YAML block (the section between the `---` markers). Look for this line:

```
author: "firstname lastname"
```

Replace `firstname lastname` with your **first and last name**, e.g.:

```
author: "Jane Doe"
```

Don't change anything else in the YAML. This is the most common place students introduce formatting errors.

The canonical pre-project YAML block (for reference and recovery if yours gets corrupted):

```yaml
---
title: "BUSN 5000 :: Pre-Project Exercise"
subtitle: Measuring Labor Force Participation
author: "First Name Last Name"
date: |
    | Summer 2026
    | (updated `r format(Sys.time(), '%d %b %y')`)
output:
  ioslides_presentation:
    css: css/project_slidedeck.css
    widescreen: false
urlcolor: "#004E60"  # Use hex code for Olympic blue from visual UGA guide
---
```

2.2.5 Step 5: Complete the code chunk on Slide 2

Slide 2 is titled "Read `LF.csv` and print its contents." The code chunk looks like:

```
lf <- read_csv(here("out", "LF.csv"))
print(__)
```

If you've never seen R code before, here's what's happening line by line:

- **Line 1.** `lf <- read_csv(here("out", "LF.csv"))` does three things at once. `read_csv("...")` is a function that reads a CSV file. `here("out", "LF.csv")` builds the file path `out/LF.csv` relative to your project root — this is why you ran `LF.R` first; it created the `LF.csv` that this line is reading. And `lf <- ...` takes the result and stores it in a new **object** named `lf`. The arrow `<-` is R's assignment operator (you'll see it used the same way most other languages use `=`).
- **Line 2.** `print(...)` prints whatever you put inside the parentheses to the slide output. The blank `__` is where you tell R *what* to print.

Replace `__` with the object you want to print. Since you just read the contents into `lf`, the answer is `lf`:

```
lf <- read_csv(here("out", "LF.csv"))
print(lf)
```

Then change the chunk option `eval = FALSE` to `eval = TRUE` at the top of the chunk. This tells R Markdown to actually run the chunk when you knit (it's set to `FALSE` by default so the template knits before you complete it).

Run the chunk (click the small green play button at the top-right of the chunk) to verify it works. You should see the `lf` data frame printed in the console — six columns named `E`, `U`, `LF`, `NILF`, `Pop`, and `LFPR`, with one row of numbers.

Write down those six numbers — you'll use them in the next step.

2.2.6 Step 6: Fill in the paragraph on Slide 3

Slide 3 is titled "Document the contents of `LF.csv`." It has a paragraph with six blanks (each blank is shown as `_____`). The paragraph reads:

> The `LF.csv` file contains the results of the labor force participation rate (LFPR) calculation using the March 2024 CPS produced by the script, `LF.R`. The LFPR (_____) is the ratio of the labor force (_____) to the population (_____). The labor force is the sum of the employed (_____) and unemployed (_____). The population is the sum of the labor force and those not in the labor force (_____). The LFPR is expressed as a percentage.

Each blank goes inside parentheses that follow a specific term — fill in the value of whatever term is named directly before the parentheses, rounded to two decimal places. For example, the first blank follows "the LFPR", so it gets the LFPR value; the second follows "the labor force", so it gets the labor force value; and so on.

For reference, here's the mapping from each blank to the matching column in your `lf` data frame:

Blank	Value from `lf`
LFPR	`LFPR`
labor force	`LF`
population	`Pop`
employed	`E`
unemployed	`U`
not in labor force	`NILF`

2.2.7 Step 7: Knit the Rmd to HTML

With your changes saved, click the **Knit** button at the top of the editor pane.

The knit produces an HTML file in your `Project/` folder. It opens automatically in the RStudio Viewer or your default browser.

2.2.8 Step 8: Compare your output to the reference

The Submission chapter includes a PDF showing exactly what each of the three pre-project slides should look like when rendered correctly — same fonts, same layout, same code chunk styling. Open it side-by-side with your knitted output and confirm they match.

If your output doesn't match exactly, something is wrong. Common causes are listed in the Common Errors chapter.

2.2.9 Step 9: Save the HTML as PDF

You don't knit-to-PDF directly. Instead, you save the HTML output as a PDF using your browser's Save As / Print to PDF function. See the Submission chapter for the exact procedure and browser-specific tips.

2.2.10 Step 10: Submit to Gradescope

Submission filename: **`firstname_lastname_pre.pdf`** (e.g., `jane_doe_pre.pdf`).

Upload to the **Preproject 0 Exercise** assignment in Gradescope.

2.3 Common pre-project mistakes

In addition to the Setup-stage mistakes in Common Errors (CSS in the wrong folder, working from Downloads, knitting before running the R script), these pre-project-specific mistakes are the ones we see most:

1. **Leaving `eval = FALSE` after completing the code chunk.** If `eval = FALSE`, the chunk doesn't run when you knit, so the output doesn't appear on the slide. Set it to `TRUE` once your code is correct.
2. **Numbers in the paragraph don't match `LF.csv`.** This usually means a student copied numbers from a peer instead of running `LF.R` themselves. The numbers you fill in must come from your own `out/LF.csv` after running your own copy of `LF.R`.
3. **YAML edits beyond the author line.** Changing anything in the YAML besides the author name will break the knit format. Don't touch the `title`, `subtitle`, `date`, `output`, `css`, or any other field.
4. **CSS not correctly placed.** If `project_slidedeck.css` is anywhere other than `Project/css/`, the knit produces an unstyled output that won't match the template. See Mistake 1 in Common Errors.
5. **Saving the wrong way.** The submission must be a PDF generated by saving the *HTML* output, not by knitting directly to PDF. See Submission for the correct workflow.

When in doubt, see Common Errors — most issues have already been documented.

2.4 What happens next

Once you've submitted the Pre-project, you've done the hardest part: your environment works. The Main Project will reuse the same `Project/` folder and the same workflow patterns. Continue to Chapter 4: Main Project when you're ready.

3Main Project

The Main Project — **Exploring Pay Differences between Women and Men** — is the central artifact of BUSN 5000. You'll use R and R Markdown to conduct an empirical analysis of the gender wage gap using the March 2024 CPS, then "knit" the code and your write-ups together into a 35-slide PDF deck.

The Project counts for **20% of your course grade** (per the syllabus).

This chapter walks through the project one slide at a time. Each slide has either a coding task, a write-up task, or both — and a point allocation. Treat it as a checklist you can work through in order.

3.1 Project-course alignment

The project is intentionally built to mirror the course content week by week. As you cover new material in class, the next chunk of the project becomes doable.

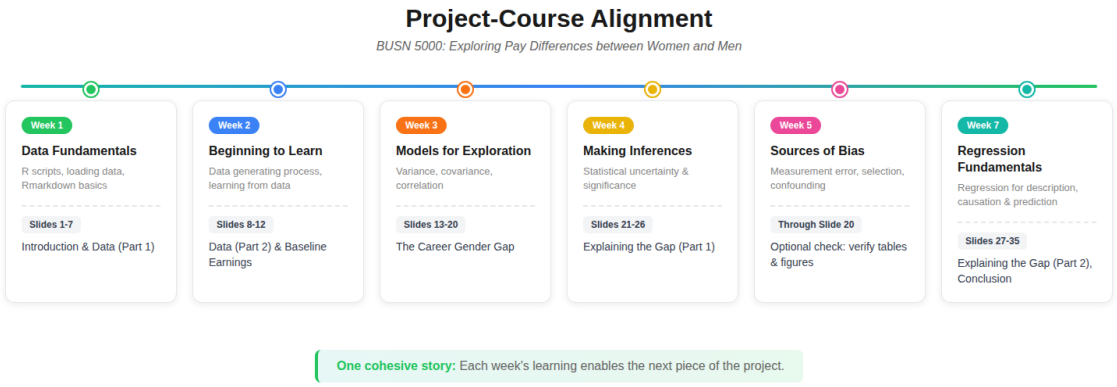

Figure 3.1: Project-course alignment

> **BUSN 5000E summer schedule**
>
> The image above shows the in-person (16-week) Spring/Fall pacing. The BUSN 5000E summer schedule is condensed — two course modules per calendar week. Re-mapped to BUSN 5000E calendar weeks:
>
> - **Week 1** (Modules 1–2, Data Fundamentals + Beginning to Learn): unlocks project slides 1–12
> - **Week 2** (Modules 3–4, Models for Exploration + Making Inferences): unlocks slides 13–20
> - **Week 3** (Modules 5–6, Measurement Error & Confounding): optional check on tables & figures through slide 20

- **Week 4** (Modules 7–8, Bayesian + Regression Fundamentals): unlocks slides 21–35; **Project Progress Check due Mon Jul 6**
- **Week 7**: **Final Project due Thu Jul 23**

If you fall behind on the course content, the project gets harder fast. Stay on schedule.

3.2 Before you start

This chapter assumes you've completed Setup and the Pre-project and that your `Project/` directory already contains the pre-project assets (CSS file, `LF.R`, raw CSVs in `data/`, etc.). The Main Project pack adds a few more files to that same directory:

- `project.Rmd` — the working file you'll edit and knit
- `cpsmar_e.R` — the R script that creates the project's data extract from the raw CPS files
- `cpsmar24_documentation.pdf` — the CPS codebook (variable definitions, value codes)
- `Project_Progress_Check_Tables_and_Figures.pdf` — reference for the progress check (Week 4)
- `project_progress_check_template.html` — a partial knit showing the project's initial tables and figures
- `check_setup_project.R` — verifies your file sort for the main project

Sort those into your existing `Project/` directory following the same conventions as the pre-project (Rmd in root, R scripts in `r/`, PDFs in root, etc.). Then run `check_setup_project.R` from RStudio (open the script, Select All, Run) to confirm everything's in place.

3.3 General rules

Before you write a single line of code, internalize these:

3.3.1 Slide format

A properly constructed slide deck has **35 slides on 35 pages of PDF output**, rendered in landscape mode, with every detail of the formatting matching the Final Project reference in the Submission chapter.

If your deck doesn't comply exactly, it will receive a 0%.

3.3.2 Write-up line limits

Each write-up slide has a specific line limit (noted in the slide's section below). **We will not read beyond the line limit.** Stay disciplined — concise writing is what's being graded, not volume.

> **!** What counts as a "line"
>
> "Lines of text" means **rendered, countable lines on your final PDF** — not sentences, and not lines as they appear in RStudio's editor or output preview. A long sentence that wraps into three visual lines on the rendered slide counts as three lines.
> Submissions that exceed the line limit are **heavily penalized**. Always check your knitted PDF and count the actual visible lines on the slide before you submit.

3.3.3 Echo settings

The template's global `echo = TRUE` setting displays your code on each slide by default. We've individually overridden this to `echo = FALSE` on a specific list of chunks where displaying the code would be redundant or take up unnecessary space:

- The `.5` chunks (`bt12.5`, `mi2.5`, `mi3.5`, `reg1.5`, `reg2.5`) — these display tables and figures from objects defined in the previous chunk; the "code" is just the object name.
- The three `datasummary` chunks (`mi1`, `ed_vars`, `demo_vars`) — `datasummary` calls are long and not informative to show.
- The appendix `var_doc_table` chunk — just renders the documentation table built in `var_doc`.

Do not edit these `echo` settings. They exist to keep the deck to exactly 35 slides.

3.3.4 HTML commands wrapping chunks

Some chunks are wrapped in `<div class="table-...">` and `</div>` tags. These are required for table formatting. **Do not edit or remove them.**

3.3.5 The YAML block

When you open `project.Rmd` for the first time, fill in your name on the `author:` line of the YAML — that's the only change you should make to the YAML. After your name is in, save the file under a new name following the convention from the Submission chapter: `firstname_lastname_project.Rmd`.

The canonical project YAML block (for reference and recovery if yours gets corrupted):

```
---
title: "BUSN 5000 Project"
subtitle: "Exploring Pay Differences between Women and Men"
author: "First Name Last Name"
date: |
    | Summer 2026
    | (updated `r format(Sys.time(), '%d %b %y')`)
output:
  ioslides_presentation:
    css: css/project_slidedeck.css
```

```
    widescreen: true
---
```

> **!** Important
>
> **Do not change anything else in the YAML.** Edits beyond the `author:` line break the rendering — see Common Errors §6.4.2 for the fix when this happens.

3.3.6 The setup chunk

The setup chunk loads the required packages and sets global options. **Do not change anything in the setup chunk.**

The default `echo = TRUE` is intentional. The `eval` toggle on each chunk is where you flip a chunk from "not run" to "run" once you've completed it — and that toggle applies only to content chunks (`btl1`, `btl2`, `mi1`, etc.), never the setup chunk. See Common Errors §6.3.3 if you've accidentally touched it.

> Workflow tip
>
> Work the project code chunks in stages. After you complete a chunk and confirm it runs cleanly (click the green play button at the top-right of the chunk), set `eval = TRUE` on that chunk so its output appears when you knit. You can run individual chunks without knitting the entire document.

3.4 A note on units

Throughout your write-ups, use **percent** and **percentage point** correctly. They are not interchangeable.

A wage gap going from 25% to 20% is a **5 percentage point** decrease — not a 5% decrease. (A 5% decrease from 25% would be 23.75%, not 20%.) Mixing the two is a recurring point of deduction.

3.5 Slide-by-slide instructions

The point allocation for each graded slide is shown in parentheses next to the slide title. Slides without a point allocation are section dividers or scaffolding chunks (the code chunks whose output is displayed on the next `.5` slide).

3.5.1 Front matter (slides 1–2)

3.5.1.1 Slide 1: BUSN 5000 Project

Title slide. It's auto-formatted with your name from the YAML `author:` line.

3.5.1.2 Slide 2: Academic Honesty Statement *(1 point)*

Indicate your compliance by typing your first and last name on the "Signature:" line. You may consult Terry Analytics Lab staff, the TA, or the instructor for help — but you may not seek assistance or otherwise collaborate with anyone else.

3.5.2 Introduction (slides 3–4)

3.5.2.1 Slide 3: Introduction

Section divider.

3.5.2.2 Slide 4: Overview *(2 points)*

Provide a brief overview of the project. Include:

- What you are trying to learn about
- The data you are using to learn
- A brief summary of your findings

Limit your overview to **6 lines of text**.

> Pro tip
>
> To quote Yoda: "Do or do not. There is no try." When you explain what your project is about or summarize what you learned, don't say "I try / seek / look / aim / attempt (or am trying / seeking / …)". Instead, say "I show / document / demonstrate / report / find …". Never say "I hope…"
> You might write this slide *last*. It's easier to summarize your findings after you've completed the analysis.

3.5.3 Data (slides 5–8)

3.5.3.1 Slide 5: Data

Section divider.

3.5.3.2 Slide 6: March 2024 CPS *(4 points)*

Using the ASEC documentation (`cpsmar24_documentation.pdf`), provide a brief overview of the March 2024 CPS. Include:

- Approximately the number of households surveyed
- A description of the standard monthly CPS
- The additional information collected in the ASEC

Limit your summary to **4 lines of text**.

> **Pro tip**
>
> Do not copy and paste from the CPS documentation. Write it in your own words.

3.5.3.3 Slide 7: March 2024 CPS Extract *(4 points)*

Complete the `read_data` code chunk to read the extract you created with `cpsmar_e.R`.

Then, in the write-up section, explain the actions you applied to the March 2024 survey in `cpsmar_e.R` to create the data extract `cpsmar_e.csv`. Include:

- The variables you selected from the person file
- The variables you selected from the household file
- The restriction(s) you applied to the data extract
- The number of observations and variables in the data extract

Limit your explanation to **6 lines of text**.

> **Pro tips**
>
> Refer to each variable by its plain English meaning, not the name used in the script. Refer to the key tidyverse "verbs" in the script, like `mutate` and `rename`.

3.5.3.4 Slide 8: Analysis sample *(2 points)*

Complete the `btl1` code chunk to create your analysis sample (`cpsmar_a`) with the following restrictions and additions:

- Restrict to individuals who are 23 to 62 years old (inclusive)
- Restrict to individuals who have positive earnings
- Create a character-valued `gender` variable

Underneath the code chunk, document the number of observations in your analysis sample.

Limit your documentation to **2 lines**.

> **Pro tips**
>
> Refer to your notes for examples of `filter` and `mutate`. Consult the Environment tab in the Northeast pane of RStudio for information about the `cpsmar_a` data frame.

3.5.4 Baseline earnings distributions (slides 9–12)

3.5.4.1 Slide 9: Baseline earnings distributions

Section divider.

3.5.4.2 Slide 10: Plotting earnings distributions

Complete the `btl2` code chunk to:

- Create the `figure1` ggplot object (the earnings distribution by gender plot)
- Calculate average earnings for each gender using the `earnings_fvm` object
- Pull the average earnings for women and men into single values (`avg_earnings_f`, `avg_earnings_m`) using `filter` and `pull`

3.5.4.3 Slide 11: Distribution of earnings by gender *(8 points)*

Complete the `btl2.5` code chunk to display Figure 1 by writing the name of the Figure 1 object.

3.5.4.4 Slide 12: Baseline comparisons *(4 points)*

Summarize the main empirical facts associated with Figure 1. Include:

- A description of the most important fact communicated by the figure
- The average earnings of men and women
- The dollar difference and percentage difference in average earnings between men and women in the sample

Limit your summary to **5 lines of text** on the slide.

> Pro tips
>
> Use inline R syntax with the `avg_earnings_f` and `avg_earnings_m` objects to insert the respective averages in your write-up (rather than typing the numbers manually).

3.5.5 The career gender gap (slides 13–20)

3.5.5.1 Slide 13: The career gender gap

Section divider.

3.5.5.2 Slide 14: Wages and hours differences *(8 points)*

Complete the `mi1` code chunk to create and display Table 1 (wages and hours by gender).

3.5.5.3 Slide 15: Documenting the differences *(2 points)*

Summarize the wage and hours differences presented in Table 1.

Limit your documentation to **3 lines of text** on the slide.

3.5.5.4 Slide 16: Plotting career log wage profiles

Complete the `mi2` code chunk to estimate log wage profiles for women and men and create Figure 2.

3.5.5.5 Slide 17: Career log wage profiles *(8 points)*

Complete the `mi2.5` chunk to display Figure 2.

3.5.5.6 Slide 18: Estimating wage differences over a career

Complete the `mi3` code chunk to create Table 2. This involves:

- Creating `males` and `females` objects using `filter` and `rename`
- Merging the two into the `diff_fvm` object using `inner_join`
- Calculating the difference between average log wages using `mutate`
- Grouping by `age_group`
- Using `kable()` to organize the results into the `table2` object

3.5.5.7 Slide 19: Evolution of the gender wage gap *(8 points)*

Complete the `mi3.5` code chunk to display Table 2.

3.5.5.8 Slide 20: Discussing the gender wage gap evolution *(2 points)*

Summarize the results presented in Figure 2 and Table 2.

Limit your summary to **3 lines of text** on the slide.

3.5.6 Explaining the gender wage gap (slides 21–30)

3.5.6.1 Slide 21: Explaining the gender wage gap

Section divider.

3.5.6.2 Slide 22: Fitting the log wage profiles

Complete the `reg1` code chunk to create Figure 3 by fitting the career profiles with a quadratic in age.

3.5.6.3 Slide 23: Log wage profiles with quadratic fits *(8 points)*

Complete the `reg1.5` chunk to display Figure 3.

3.5.6.4 Slide 24: Gender differences in education *(8 points)*

Complete the `ed_vars` code chunk to create and display Table 3 (educational attainment by gender).

3.5.6.5 Slide 25: Gender differences in demographics *(8 points)*

Complete the `demo_vars` code chunk to create and display Table 4 (demographic characteristics by gender).

3.5.6.6 Slide 26: Documenting differences in characteristics *(4 points)*

- Summarize the educational attainment differences presented in Table 3
- Summarize the differences in demographic characteristics presented in Table 4

Limit your documentation to **6 lines of text** on the slide.

3.5.6.7 Slide 27: Controlling for education and demographic characteristics

Complete the `reg2a` code chunk to create:

- The `singles` subset of the analysis data
- The `models` object containing five regression models that incrementally add controls for education and demographic characteristics, with the fifth restricting to unmarried workers without children under 6 ("Only Singles")

In the regression analysis, you'll distinguish between personal and household characteristics among the demographic variables.

> Pro tips
>
> - Moving from top to bottom in the `models` object, the list of controls grows as indicated by the name of the model — with the exception of the final model "Only Singles", which estimates the same relationship as the "Add Person" model but on the new subset.
> - To decide whether a variable belongs in the "Add Person" or "Add Household" model, ask yourself: *"Can I define this variable for a particular individual irrespective of other individuals who may or may not exist in their life?"* If yes → Person. If no (it depends on someone else, like a spouse or a child) → Household.

3.5.6.8 Slide 28: Reporting the results

Complete the `reg2b` code chunk to create Table 5. This involves:

- Constructing the coefficient map object to display only the gender and age coefficient estimates
- Constructing the goodness-of-fit object to display sample size and R^2 values
- Constructing a `rows` object to distinguish the regression specification associated with each column

> **Pro tip**
>
> Make sure you report robust standard errors and indicate that you do in a table note.

3.5.6.9 Slide 29: Explaining the gender wage gap *(8 points)*

Complete the `reg2.5` chunk to display Table 5.

3.5.6.10 Slide 30: Documenting the findings *(4 points)*

Summarize how the estimated average gender wage gap changes as you add education, personal, and household characteristics to the regression — and then how it changes when the sample is restricted to singles.

Limit your documentation to **6 lines of text** on the slide.

> **Pro tips**
>
> Start your write-up with the baseline model and describe subsequent results relative to the baseline. Focus on the Female coefficient estimate and its standard error. Remember how the coefficient estimate is correctly interpreted.

3.5.7 Conclusion (slides 31–32)

3.5.7.1 Slide 31: Conclusion

Section divider.

3.5.7.2 Slide 32: Summary *(4 points)*

Briefly summarize the objective of the project and its main findings. Note:

- The sample on which your analysis is based
- The overall gender wage gap
- How it evolves over a career
- How it varies when controlling for education and demographic characteristics

Limit your documentation to **7 lines of text** on the slide.

3.5.8 Appendix (slides 33–35)

3.5.8.1 Slide 33: Appendix

Section divider.

3.5.8.2 Slide 34: Data documentation

Complete the `var_doc` code chunk to create a table of the main variables used in this project with their definitions.

3.5.8.3 Slide 35: List of main variables with definitions *(4 points)*

Once you complete the `var_doc` chunk on the previous slide, set `eval = TRUE` on this slide to render the table of main variables with their definitions.

3.6 Submission

See the Submission chapter for:

- Filename convention (`firstname_lastname_project.pdf`)
- The HTML → PDF workflow
- Browser-specific save-as-PDF settings
- The 35-slide visual reference

3.7 Common pitfalls

The recurring failure modes for the Main Project are catalogued in Common Errors. The ones most specific to the Main Project (vs. the Pre-project) are:

- **Variable name confusion** — defining `cpsmar_a` but referencing `cpsmar_e` (or vice versa) in a later chunk. See Common Errors §6.3.2.
- **Forgetting to run `cpsmar_e.R` before knitting.** See Common Errors §6.3.1.
- **Touching the setup chunk's `include = FALSE`** — same scaffolding error as the pre-project. See Common Errors §6.3.3.
- **Editing the YAML beyond the author line.** See Common Errors §6.4.2.

When in doubt, see Common Errors — most issues have already been documented.

3.8 What happens next

- For the optional Project Progress Check at Week 4, see Chapter 5: Progress Check.
- When the Final Project deadline arrives, see Chapter 6: Submission for the save-as-PDF walkthrough and the filename convention.

4Progress Check

The Project Progress Check is an **optional** mid-project submission that gives you a small bonus on your Project grade in exchange for showing that the core tables and figures are on track. It's offered in Week 4 of the semester, halfway between the Pre-project and the Final Project.

You don't have to submit it. We strongly recommend that you do.

4.1 Why bother

Two reasons:

1. **Bonus points.** A complete and correct progress check earns up to **5 bonus points** on your Project grade (per the syllabus).
2. **A safety net for the Pre-project.** If you earned a 0 on the Pre-project, the progress check is your one opportunity to earn those points back. Submit a clean progress check and the lost pre-project points are recovered.

Even if you passed the Pre-project, treating the progress check as required is the safer move — it forces you to confirm your formatting is right and your core figures are working well before the final deadline.

4.2 What you'll do

The progress check is not a separate file or a separate document. You continue working on the same `project.Rmd` you started after finishing the Pre-project. On the progress check deadline, you save the in-progress `.Rmd` as a PDF (using the same workflow as the Pre-project and Final Project — see Submission) and upload it to the Project Progress Check assignment on Gradescope.

There is no "progress check version" of the project. There's just the project, as it stands at the progress-check deadline.

4.3 What's graded

The progress check covers **about two-thirds of the project's core tables and figures**. Specifically, we look at six slides:

Slide	Title	What we check
1	Title slide	Your name filled in (and overall formatting is correct)
2	Academic Honesty Statement	Your name on the Signature line
11	Distribution of earnings by gender	Figure 1 is visible, completed, and correct

Slide	Title	What we check
14	Wages and hours differences	Table 1 is visible, completed, and correct
17	Career log wage profiles	Figure 2 is visible, completed, and correct
19	Evolution of the gender wage gap	Table 2 is visible, completed, and correct

> **Note**
>
> The slide numbers above match the canonical 35-slide layout documented in the Final Project reference in the Submission chapter. Your deck must be 35 slides total even if many of the content slides are still placeholder.

4.4 Rubric

Three possible outcomes:

Result	Score	Criteria
Full credit	5 bonus points	All six graded slides are visible, completed, and correct, AND the deck is 100% compliant with the formatting requirements.
Partial credit	3 bonus points	The deck is 100% compliant with formatting, but at least one graded slide has a content error.
No credit	0 bonus points	Any formatting error (wrong CSS, wrong orientation, knit-to-PDF instead of HTML→PDF, etc.).

> **Important**
>
> Formatting errors are a hard zero. A perfectly correct content submission that's saved with the wrong orientation, lost CSS, or the wrong PDF workflow earns 0. Formatting first, then content.

4.5 What's *not* graded

- **Written sections.** We do not read the prose write-ups (Overview, Baseline comparisons, Documenting the differences, etc.) during the progress check. Those are evaluated only in

the Final Project submission.
- **Other tables and figures.** Slides 23, 24, 25, 29 (Table 3, Table 4, Figure 3, Table 5) are not part of the progress check. They're evaluated in the Final Project.

This focus is intentional — the progress check is a checkpoint, not a full grade. Use the un-graded slides for trying things out, sketching write-ups, etc.

4.6 Submission

- **Filename:** `firstname_lastname_progress.pdf` (e.g., `jane_doe_progress.pdf`).
- **Format:** 35-slide PDF, landscape, formatting compliant with the knitted reference in the Submission chapter.
- **Deadline:** As listed on the course schedule. Late submissions are not accepted.
- **Gradescope assignment:** "Project Progress Check 0".

The save-as-PDF workflow is identical to the Pre-project and Final Project. See Submission for browser-by-browser print dialog settings.

4.7 Common pitfalls

These are the recurring failure modes from past semesters:

1. **Touching the setup chunk.** Do not change `include = FALSE` to `include = TRUE` on the setup chunk. The `FALSE`/`TRUE` toggle only applies to editing the `eval =` argument in the *content* code chunks (`btl1`, `btl2`, `mi1`, etc.), never the setup chunk.
2. **Skipping the extract script.** You must run `cpsmar_e.R` (and have the pre-project data files carried over from the Pre-project) before knitting. The `.Rmd` reads the extract it produces.
3. **Variable name confusion.** A common error: you save the analysis sample as `cpsmar_a` but then reference `cpsmar_e` in a later chunk (or vice versa). Read your code carefully and make sure each chunk references the right data frame.
4. **CSS in the wrong folder.** Same as the Pre-project — `project_slidedeck.css` must be in `Project/css/`. See Common Errors for diagnosis.
5. **Saving as PDF the wrong way.** Same as the Pre-project — you must knit to HTML first, then save the HTML as a PDF via your browser's print dialog (landscape orientation, background graphics enabled). Knitting directly to PDF strips the CSS formatting and earns a 0. See Submission for the per-browser walkthroughs.

4.8 What happens next

After the progress check, you continue working on `project.Rmd` toward the Final Project deadline. The work you did for the progress check stays in place — you fill in the remaining tables, figures, and write-ups, then submit the complete deck as `firstname_lastname_project.pdf`. See Chapter 4: Main Project for the slide-by-slide walkthrough and Submission for the final submission mechanics.

5Submission

This chapter covers how to submit your work to Gradescope without losing points to formatting. The same workflow applies to all three project artifacts you submit: the **Pre-project**, the optional **Project Progress Check**, and the **Final Project**.

In the past four years, the vast majority of pre-projects, project progress checks, and final projects that earned a 0 were the result of something done wrong here.

5.1 Filename conventions

Each of the three project artifacts has a required filename. Use **lowercase** for first and last name, **underscores** between words, and the `.pdf` extension.

Artifact	Filename pattern	Example
Pre-project	`firstname_lastname_pre.pdf`	`jane_doe_pre.pdf`
Project Progress Check	`firstname_lastname_progress.pdf`	`jane_doe_progress.pdf`
Final Project	`firstname_lastname_project.pdf`	`jane_doe_project.pdf`

> **!** Important
>
> Use the filename pattern exactly as written. It keeps your three submissions distinct in your project folder (so a later one doesn't overwrite an earlier one), and it makes each artifact easy to locate when you need to refer back to it. Required, not optional.

5.2 Late submission policy

Per the syllabus: **late submissions will not be accepted** for the Pre-project, the Project Progress Check, or the Final Project. The relevant deadlines are on the course schedule. There are no extensions and no late-credit options.

5.3 The submission workflow at a glance

Each submission goes through the same three steps:

1. **Knit** your `.Rmd` to HTML inside RStudio (the **Knit** button in the editor pane).

   > **!** Important
   >
   > Click the **Knit** button itself, not the small dropdown arrow next to it. The dropdown offers "Knit to HTML / PDF / Word" options that override the output format we've configured in the Rmd's YAML — picking one will overwrite our formatting and your

output will be rejected.

2. **Save** the resulting HTML page as a PDF using your browser's print-to-PDF function (instructions per browser below).
3. **Upload** the PDF to the corresponding Gradescope assignment under the correct filename.

⚠ Do not knit directly to PDF

Knitting straight to PDF from R Markdown uses a different rendering path (LaTeX) that strips the CSS-based slide formatting we use. The output will not match the reference and **will receive a 0%** as a formatting violation.
You **must** knit to HTML, then save the HTML as PDF.

5.4 Saving HTML as PDF

After knitting, the HTML output opens in your browser (or in RStudio's Viewer pane — if it opens there, click the small icon at the top of the Viewer to open it in your default browser).

In every browser, the print dialog is what you use to save as PDF. The settings you need are roughly the same across browsers:

- **Destination:** "Save as PDF"
- **Layout / Orientation: Landscape**
- **Color:** Color (not Black and White)
- **Margins:** Default or None
- **Pages per sheet:** 1
- **Background graphics: Enabled** (this is the easy-to-miss one — without it, the colored UGA styling disappears)

Per-browser walkthroughs below.

5.4.1 Google Chrome (and Microsoft Edge)

Edge uses the same print dialog as Chrome — these steps work for both.

1. Click the three vertical dots in the top-right of the browser.
2. Click **Print**.
3. Match the settings shown below — most importantly, **Destination = Save as PDF**, **Layout = Landscape**, and **More settings → Background graphics = checked**.

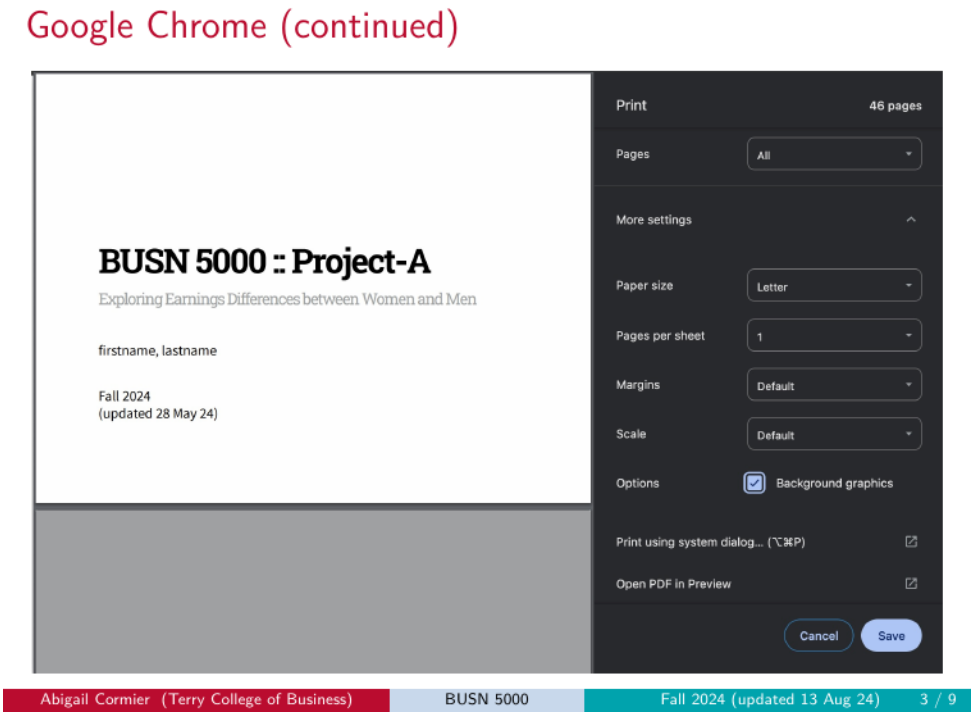

Figure 5.1: Chrome print dialog settings

4. Click **Save**. Use the correct filename when prompted (see Filename conventions).

5.4.2 Firefox

1. Click the three horizontal lines (the “hamburger” menu) in the top-right of the browser.
2. Click **Print**.
3. Match the settings shown below across the two screens. Key settings: **Destination = Save to PDF**, **Orientation = Landscape**, and **Options → Print backgrounds = checked**.

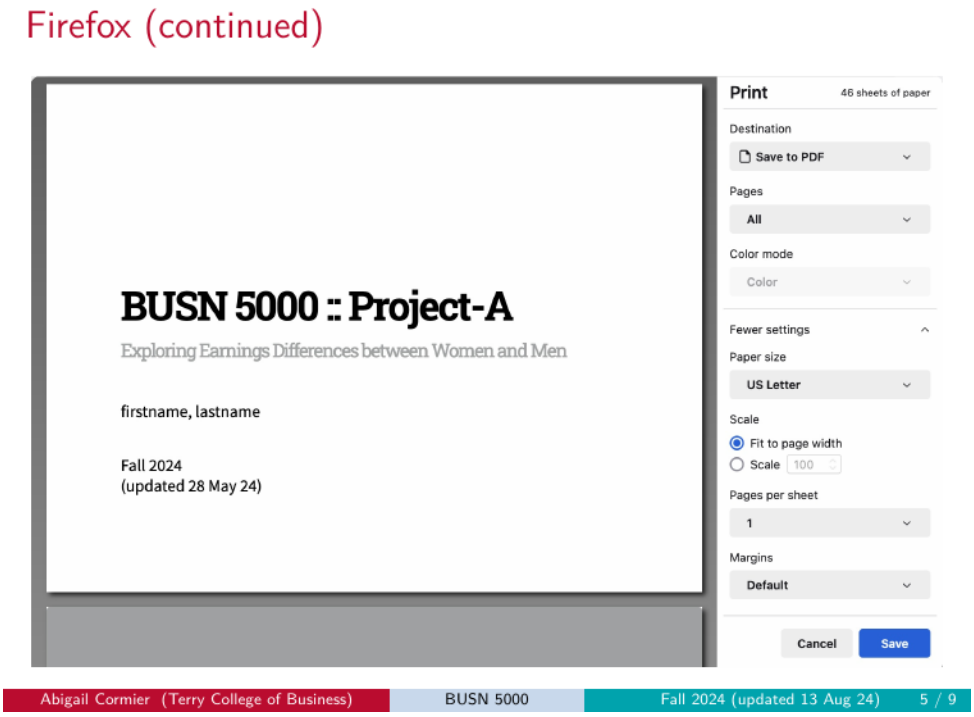

Figure 5.2: Firefox print settings — page 1

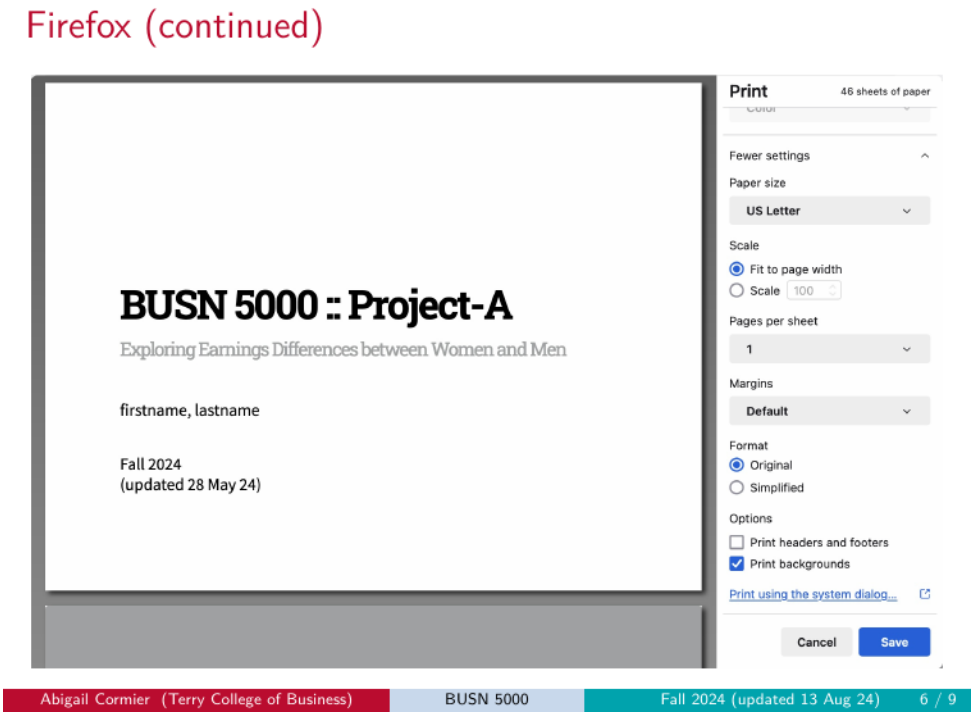

Figure 5.3: Firefox print settings — page 2

4. Click **Save** and use the correct filename.

5.4.3 Safari

1. Click **File** in the top-left menu bar.
2. Click **Print**.
3. Match the settings shown below across the two screens. Key settings: **Orientation = Landscape**, **Print backgrounds = checked**, and the **PDF dropdown in the bottom-left** must be set to **Save as PDF** (not Print).

Safari (continued)

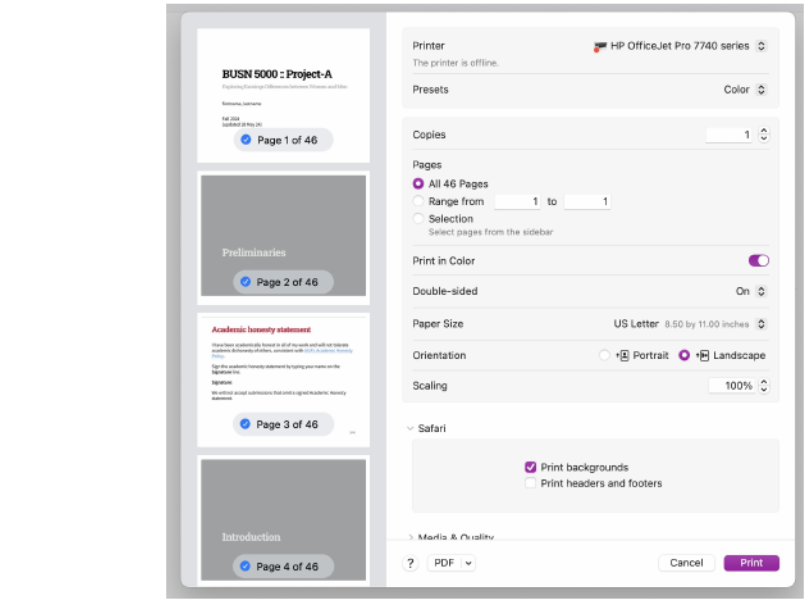

Figure 5.4: Safari print settings — page 1

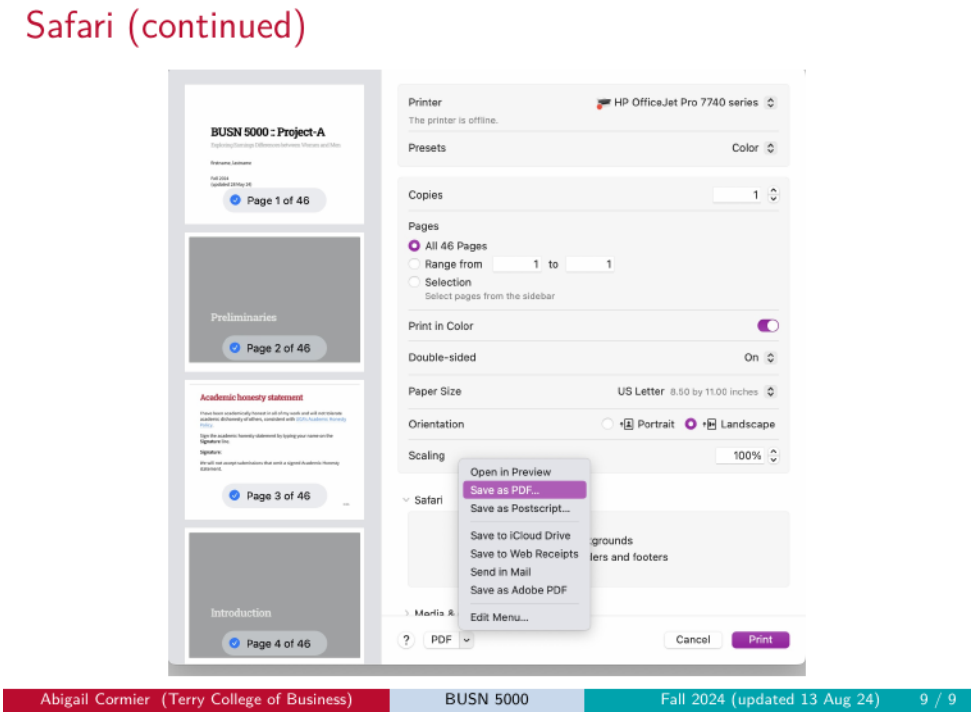

Figure 5.5: Safari print settings — page 2

4. Click **Save** and use the correct filename.

5.5 Pre-project reference: what your three slides should look like

Below are the three pre-project slides rendered correctly. Your knitted PDF should match these for **formatting** (fonts, colors, slide layout, code chunk styling). The **content** will differ — your name will be on the title slide and the numbers in your print-out and your fill-ins will reflect your `LF.csv`.

> **Note**
>
> The images below show legacy examples (date stamp from a prior semester, comma in the name placeholder). They are reference for the slide *format*, not the exact text content. Your version will show Summer 2026, your name without a comma, and the current CPS numbers.

5.5.1 Reference slide 1 — Title slide

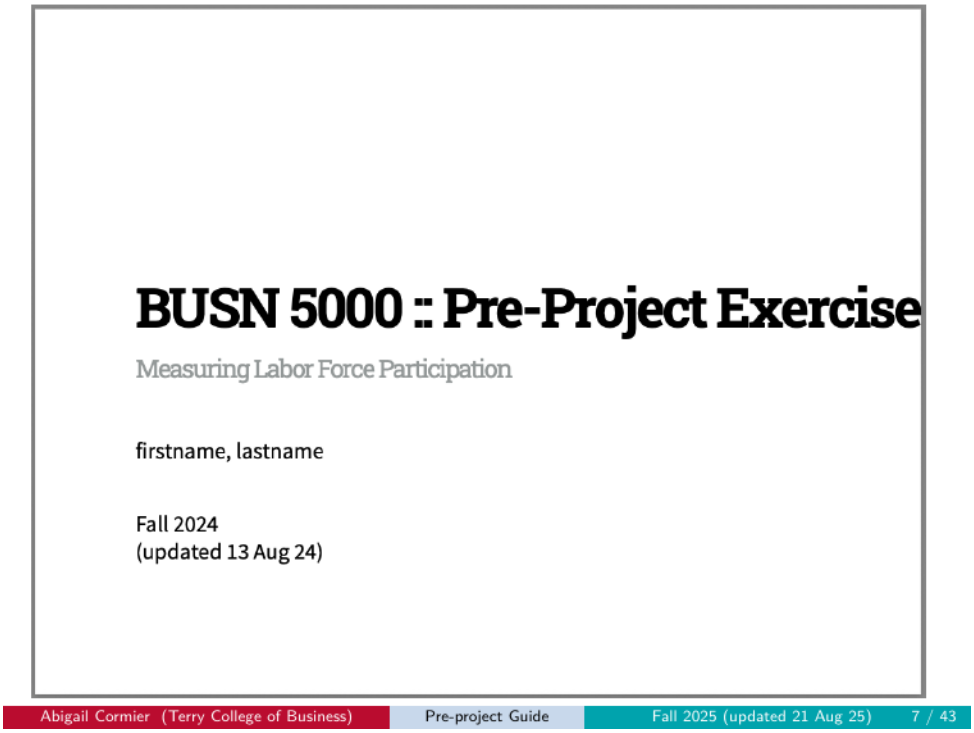

Figure 5.6: Title slide reference

What yours should look like:

- Title and subtitle as shown
- Your first and last name on the author line
- Summer 2026 date

5.5.2 Reference slide 2 — Read `LF.csv` and print its contents

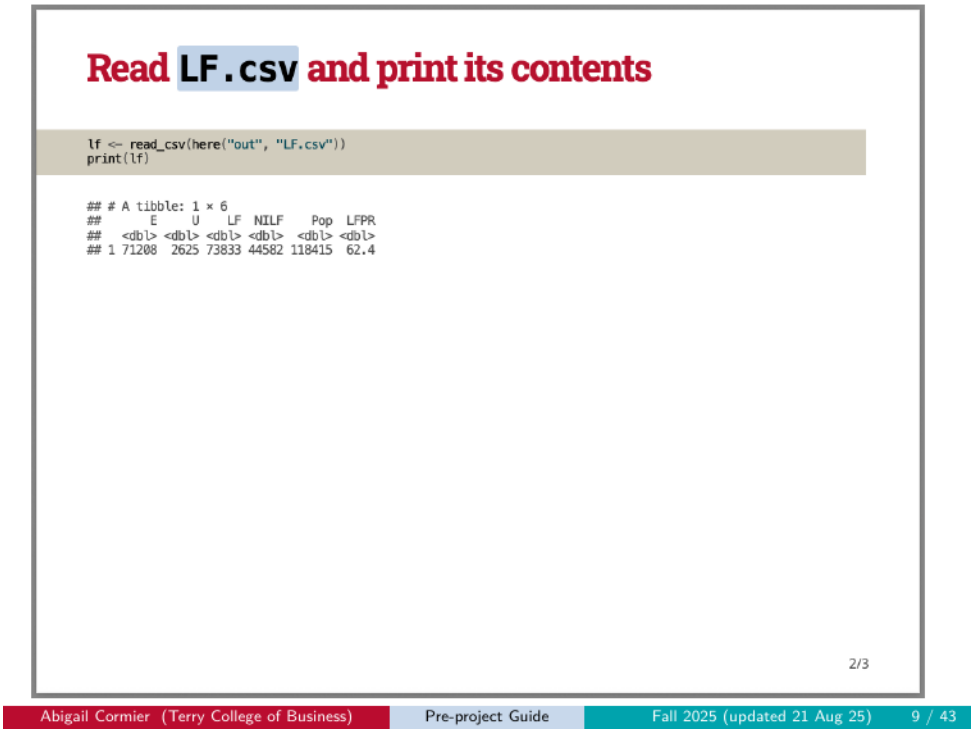

Figure 5.7: LF print-out slide reference

What yours should look like:

- The code chunk visible (formatted with the UGA code-block style)
- The `lf` data frame printed below it with six named columns

5.5.3 Reference slide 3 — Document the contents of `LF.csv`

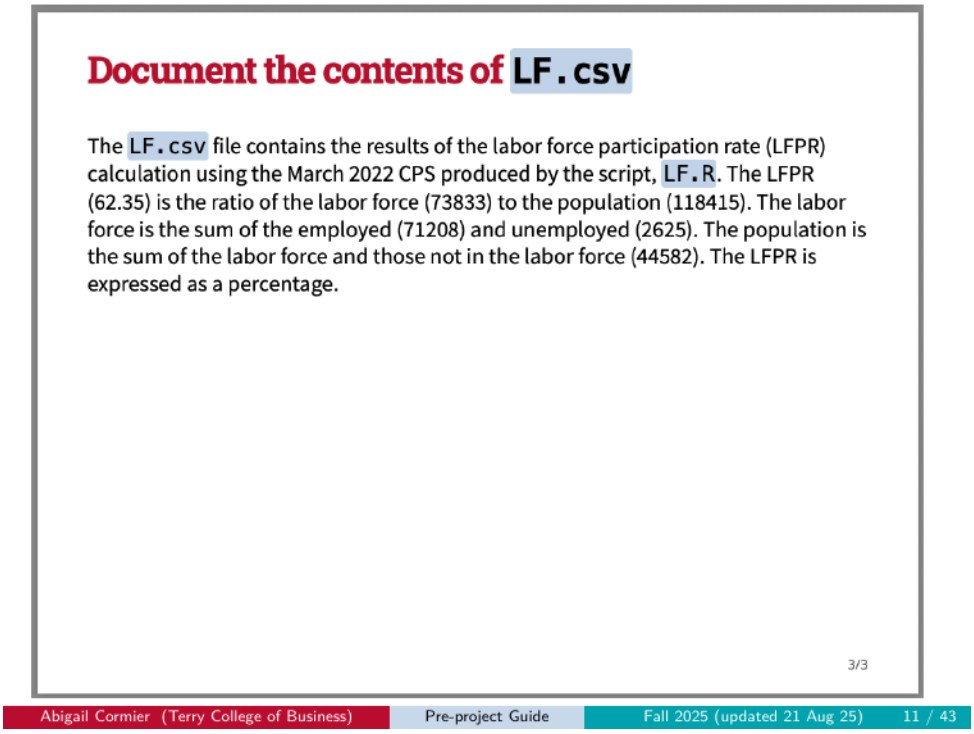

Figure 5.8: Communication slide reference

What yours should look like:

- The paragraph fully filled in, no remaining `_____` blanks
- Values rounded to two decimal places
- Numbers matching your `out/LF.csv` (not numbers copied from this reference)

5.6 Final Project reference: what your 35 slides should look like

Below are the 35 final-project slides rendered correctly. Your knitted PDF should match these for **formatting** (fonts, colors, slide layout, tables, figures). The **content** will reflect your own work — your name on the title slide, your written sections, your data outputs, and your figures.

> **Note**
>
> The images below show legacy examples (date stamp from a prior semester). The format is what to match. Your version will show Summer 2026 and current CPS numbers.

5.6.1 Front matter (slides 1–2)

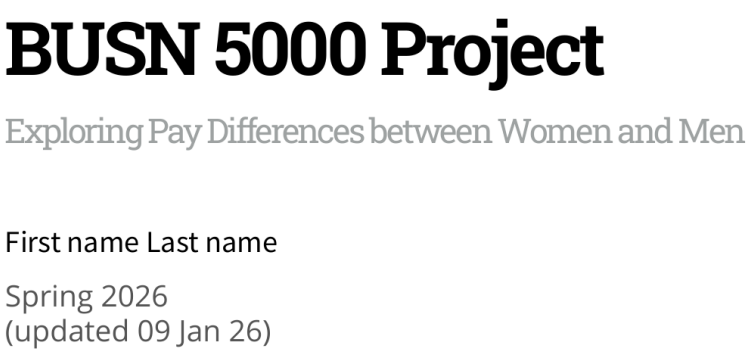

Figure 5.9: Slide 1: Title

Academic honesty statement

I have been academically honest in all of my work and will not tolerate academic dishonesty of others, consistent with UGA's Academic Honesty Policy.

Sign the academic honesty statement by typing your name on the **Signature** line.

Signature:

We will not accept submissions that omit a signed Academic Honesty statement.

2/35

Figure 5.10: Slide 2: Academic Honesty Statement

5.6.2 Introduction (slides 3–4)

Figure 5.11: Slide 3: Introduction (section divider)

Overview

Insert your content here replacing these words with your own text.

4/35

Figure 5.12: Slide 4: Overview

5.6.3 Data (slides 5–8)

Figure 5.13: Slide 5: Data (section divider)

March 2024 CPS

Insert your content here replacing these words with your own text.

6/35

Figure 5.14: Slide 6: March 2024 CPS

March 2024 CPS Extract

```
cpsmar_e <- _____(here("_____", "_____"))
```

Insert your content here replacing these words with your own text.

7/35

Figure 5.15: Slide 7: March 2024 CPS Extract

Analysis sample

```
cpsmar_a <- cpsmar_e %>%
  _____(
    age >= _____,
    age <= _____,
    earnings > _____
  ) %>%
  _____(
    gender = _____(female == _____, "Female", "Male"),
    wage = _____/(_____*_____),
    lwage = log(_____),
    Black = case_when(race==_____~1, TRUE ~ 0),
    south = case_when(region==_____~1, TRUE ~ 0),
    married = case_when((marital==_____ | marital==_____ | marital==_____)~1, TRUE ~ 0),
    age_centered = age - 23
  )
```

Insert your content here replacing these words with your own text.

8/35

Figure 5.16: Slide 8: Analysis sample

5.6.4 Baseline earnings distributions (slides 9–12)

Figure 5.17: Slide 9: Baseline earnings distributions (section divider)

Plotting earnings distributions

```
figure1 <- ggplot(_____, aes(x = _____, group = _____, fill = _____)) +
  geom_density(alpha = 0.4) +
  labs(
    title="Figure 1. Distribution of earnings by gender",
    x="Earnings",
    y="Density"
    )+
  theme_minimal()
earnings_fvm <- cpsmar_a %>%
  group_by(_____) %>%
  summarize(avg_earnings = round(_____(_____, na.rm = TRUE),0))

avg_earnings_f <- earnings_fvm %>%
  _____(gender == "Female") %>%
  pull(avg_earnings) # `pull` extracts the "avg_earnings" value for "Female" from earnings_fvm, a single value since the data
only record two genders.

avg_earnings_m <- earnings_fvm %>%
  _____(gender == "Male") %>%
  pull(avg_earnings) # `pull` extracts the "avg_earnings" value for "Male" from earnings_fvm, a single value since the data only
record two genders.
```

10/35

Figure 5.18: Slide 10: Plotting earnings distributions

Distribution of earnings by gender

11/35

Figure 5.19: Slide 11: Distribution of earnings by gender (Figure 1)

Baseline comparisons

Insert your content here replacing these words with your own text.

12/35

Figure 5.20: Slide 12: Baseline comparisons

5.6.5 The career gender gap (slides 13–20)

Figure 5.21: Slide 13: The career gender gap (section divider)

Figure 5.22: Slide 14: Wages and hours differences (Table 1)

Documenting the differences

Insert your content here replacing these words with your own text.

15/35

Figure 5.23: Slide 15: Documenting the differences

Plotting career log wage profiles

```
cef_fvm_w <- cpsmar_a %>%
  _____(_____, _____) %>%
  _____(avg_lwage = mean(_____, na.rm = TRUE))

figure2 <- ggplot(_____, aes(x = _____, y = _____, color = _____, linetype = gender, linewidth = gender)) +
  geom_point() +
  geom_line() +
  scale_linetype_manual(values = c("Female" = "longdash", "Male" = "solid")) +
  scale_linewidth_manual(values = c("Female" = 0.7, "Male" = 0.5)) +
  guides(linewidth = "none") +
  labs(
    title="Figure 2. Career log-wage profiles for women and men",
    x="Year",
    y="Average log wage"
    )+
  theme_minimal()
```

16/35

Figure 5.24: Slide 16: Plotting career log wage profiles

Career log wage profiles

17/35

Figure 5.25: Slide 17: Career log wage profiles (Figure 2)

Estimating wage differences over a career

```
males <- cef_fvm_w %>%
  _____(gender == "Male") %>%
  _____(avg_lwage_male = avg_lwage) %>%
  _____(-gender)
females <- cef_fvm_w %>%
  _____(gender == "Female") %>%
  _____(avg_lwage_female = avg_lwage) %>%
  _____(-gender)

diff_fvm <- inner_join(males, females, by = "age_centered") %>%
  _____(age_centered <= 30) %>%
  _____(
    diff = avg_lwage_male - avg_lwage_female,
    age_group = cut(
      age_centered,
      breaks = c(-1, 10, 20, 30),
      labels = c("1-10", "11-20", "21-30"))
    ) %>%
  group_by(_____) %>%
  _____(mean_diff = mean(_____)*100)

table2 <- kable(
  diff_fvm,
  digits = 2,
  col.names = c("Year Range", "Avg Pct Difference"),
  align = "cc",
  caption = "Table 2. Percent wage differences, first 30 years",
  ) %>%
  kable_styling(position = "center")
```

18/35

Figure 5.26: Slide 18: Estimating wage differences over a career

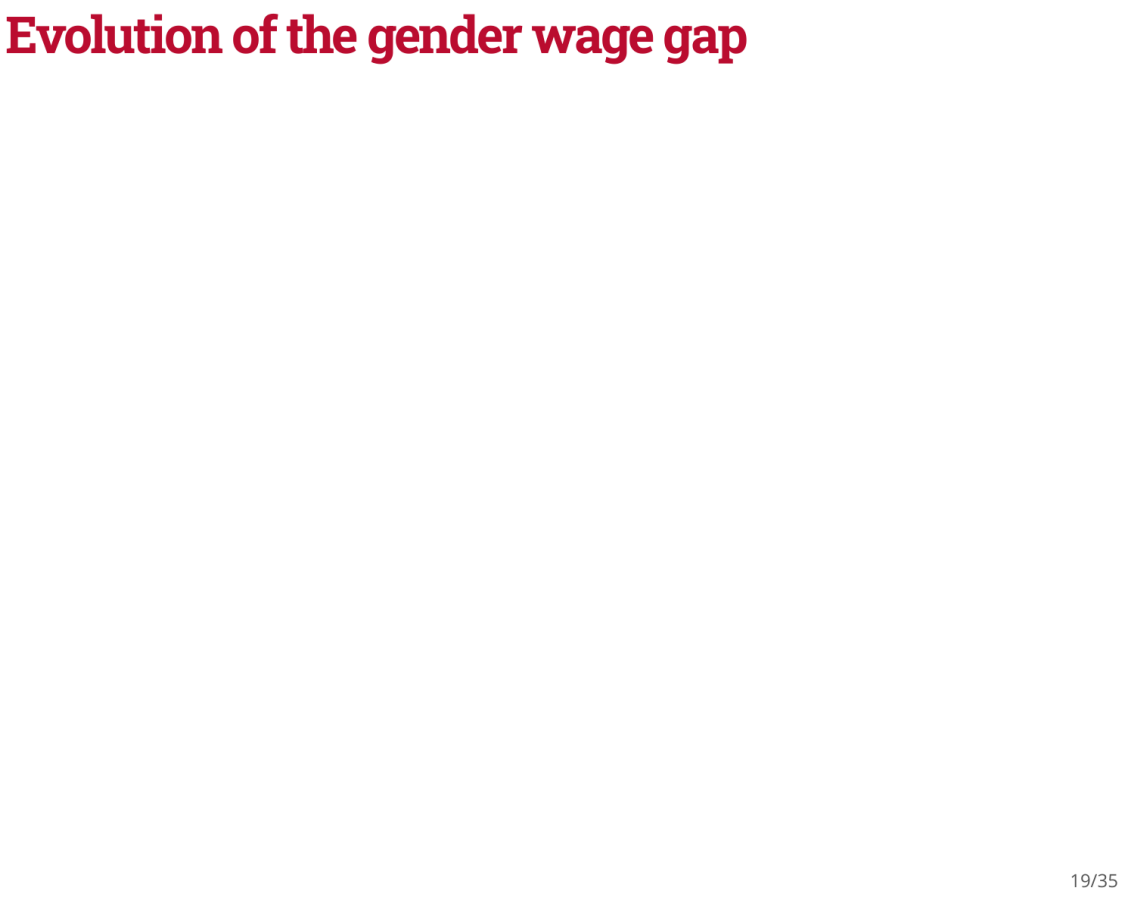

Figure 5.27: Slide 19: Evolution of the gender wage gap (Table 2)

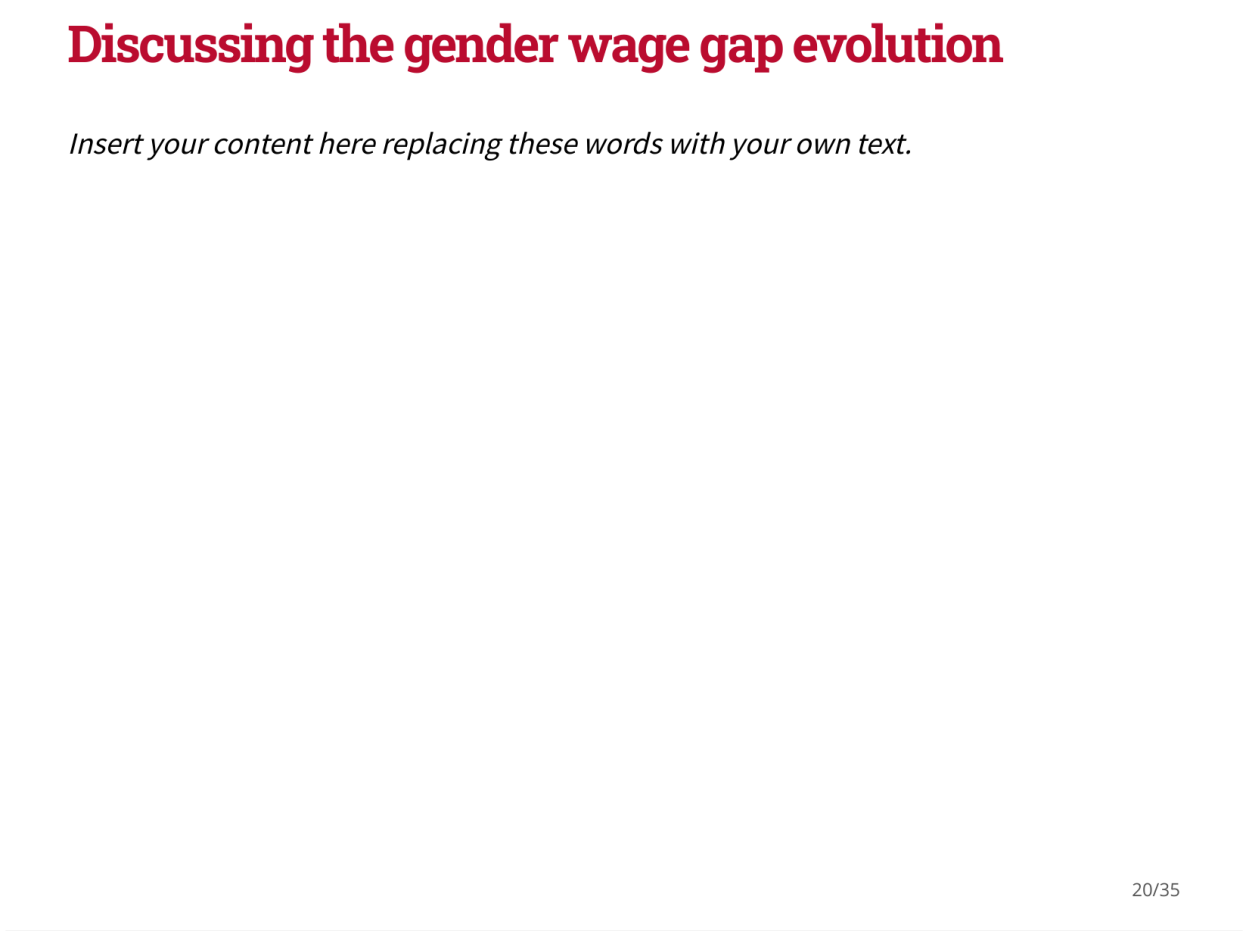

Figure 5.28: Slide 20: Discussing the gender wage gap evolution

5.6.6 Explaining the gender wage gap (slides 21–30)

Figure 5.29: Slide 21: Explaining the gender wage gap (section divider)

Fitting the log wage profiles

```
formula <- _____ ~ _____ + I(_____^2)
figure3 <- figure2 +
  geom_smooth(
    method = _____,
    formula = _____,
    aes(group = _____),
    se = FALSE
    ) +
  stat_poly_eq(
    aes(label =  after_stat(eq.label)),
    formula = _____,
    parse = TRUE
    ) +
  labs(
    title="Figure 3. Career log-wage profiles with quadratic fits",
    x="Year",
    y="Average log wage"
    )+
  theme_minimal()+
  theme(legend.position = "bottom")
```

22/35

Figure 5.30: Slide 22: Fitting the log wage profiles

Log wage profiles with quadratic fits

23/35

Figure 5.31: Slide 23: Log wage profiles with quadratic fits (Figure 3)

Gender differences in education

24/35

Figure 5.32: Slide 24: Gender differences in education (Table 3)

Gender differences in demographics

25/35

Figure 5.33: Slide 25: Gender differences in demographics (Table 4)

Documenting differences in characteristics

Insert your content here replacing these words with your own text.

26/35

Figure 5.34: Slide 26: Documenting differences in characteristics

Controlling for education and demographic characteristics

```
singles <- cpsmar_a %>%
  _____(
    married==_____,
    child_u6==_____
    )
models <- list(
  "Baseline"      = lm(_____ ~ _____ +
                       age_centered + I(age_centered^2),
                       _____),
  "Add Education" = lm(_____ ~ _____ +
                       age_centered + I(age_centered^2) + _____ + _____ + _____,
                       _____),
  "Add Person"    = lm(_____ ~ _____ +
                       age_centered + I(age_centered^2) + _____ + _____ + _____ +
                       _____ + _____ + _____ + _____,
                       _____),
  "Add Household" = lm(_____ ~ _____ +
                       age_centered + I(age_centered^2) + _____ + _____ + _____ +
                       _____ + _____ + _____ + _____ +
                       _____ + _____,
                       _____),
  "Only Singles"  = lm(_____ ~ _____ +
                       age_centered + I(age_centered^2) + _____ + _____ + _____ +
                       _____ + _____ + _____ + _____,
                       _____)
)
```

27/35

Figure 5.35: Slide 27: Controlling for education and demographic characteristics

Reporting the results

```
cm <- c(
  'female'           = 'Female',
  'age_centered'     = 'Age',
  'I(age_centered^2)' = 'Age$^2$',
  '(Intercept)'      = 'Constant'
)
gm <-  tibble::tribble(
  ~raw, ~clean, ~fmt,
  "nobs", "$N$", 0,
  "r.squared", "$R^2$", 2
)
rows <- tribble(~term, ~Baseline, ~Add_Education, ~Add_Person, ~Add_Household, ~Only_Singles,
  'Education controls',    ' ',         'X',           'X',          'X',           'X',
  'Demographic controls', ' ',          ' ',           'X',          'X',           'X',
  'Household controls',    ' ',         ' ',           ' ',          'X',           'X'
)
attr(rows, 'position') <- c(9, 10, 11) # Positions where you want these rows to appear

table5 <- modelsummary(
  models,
  add_rows = rows,
  coef_map = _____,
  gof_map = _____,
  vcov = c("_____","_____","_____","_____","_____"),
  title = "Table 5. OLS estimates of the gender wage gap",
  notes = "_____ standard errors in parentheses.",
  escape = FALSE
  )
```

28/35

Figure 5.36: Slide 28: Reporting the results

Explaining the gender wage gap

29/35

Figure 5.37: Slide 29: Explaining the gender wage gap (Table 5)

Documenting the findings

Insert your content here replacing these words with your own text.

30/35

Figure 5.38: Slide 30: Documenting the findings

5.6.7 Conclusion (slides 31–32)

Figure 5.39: Slide 31: Conclusion (section divider)

Summary

Insert your content here replacing these words with your own text.

32/35

Figure 5.40: Slide 32: Summary

5.6.8 Appendix (slides 33–35)

Figure 5.41: Slide 33: Appendix (section divider)

Data documentation

```r
# Define the variables and their descriptions
variables <- data.frame(
  Variable = c(
    "age",
    "earnings",
    "hours",
    "race",
    "marital",
    "HSGrad",
    "SomeColl",
    "CollDeg",
    "region",
    "female",
    "hisp",
    "fulltime"
    ),
  Definition = c(
    "years; capped at 85",
    "_____",
    "_____",
    "respondent's race (1 = White only, 2 = Black only, 3 = AI only, 4 = Asian only, 5 = Hawaiian/Pacific Islander only (HP), 6 = White-Black, 7 = White-AI, 8
= White-Asian, 9 = White-HP, 11 = Black-Asian, 12 = Black-HP, 13 = AI-Asian, 14 = AI-HP, 15 = Asian-HP, 16 = White-Black-AI, 17 = White-Black-Asian, 18 =
White-Black-HP, 19 = White-AI-Asian, 20 = White-AI-HP, 21 = White-Asian-HP, 22 = Black-AI-Asian, 23 = White-Black-AI-Asian, 24 = White-AI-Asian-HP, 25 = White-
Black-AI-Asian-HP, 25 = Other 3 race comb., 26 = Other 4 or 5 race comb.)",
    "marital status (1 = Married civilian, 2 = Married AF, 3 = Married absent, 4 = Widowed, 5 = Divorced, 6 = Separated, 7 = Never married)",
    "= 1, if _____",
    "= 1, if _____",
    "= 1, if _____",
    "household region (1 = Northeast, 2 = Midwest, 3 = South, 4 = West)",
    "= 1 if _____",
    "= 1 if Hispanic, Spanish, or Latino",
    "= 1 if _____"
  )
)
```

34/35

Figure 5.42: Slide 34: Data documentation

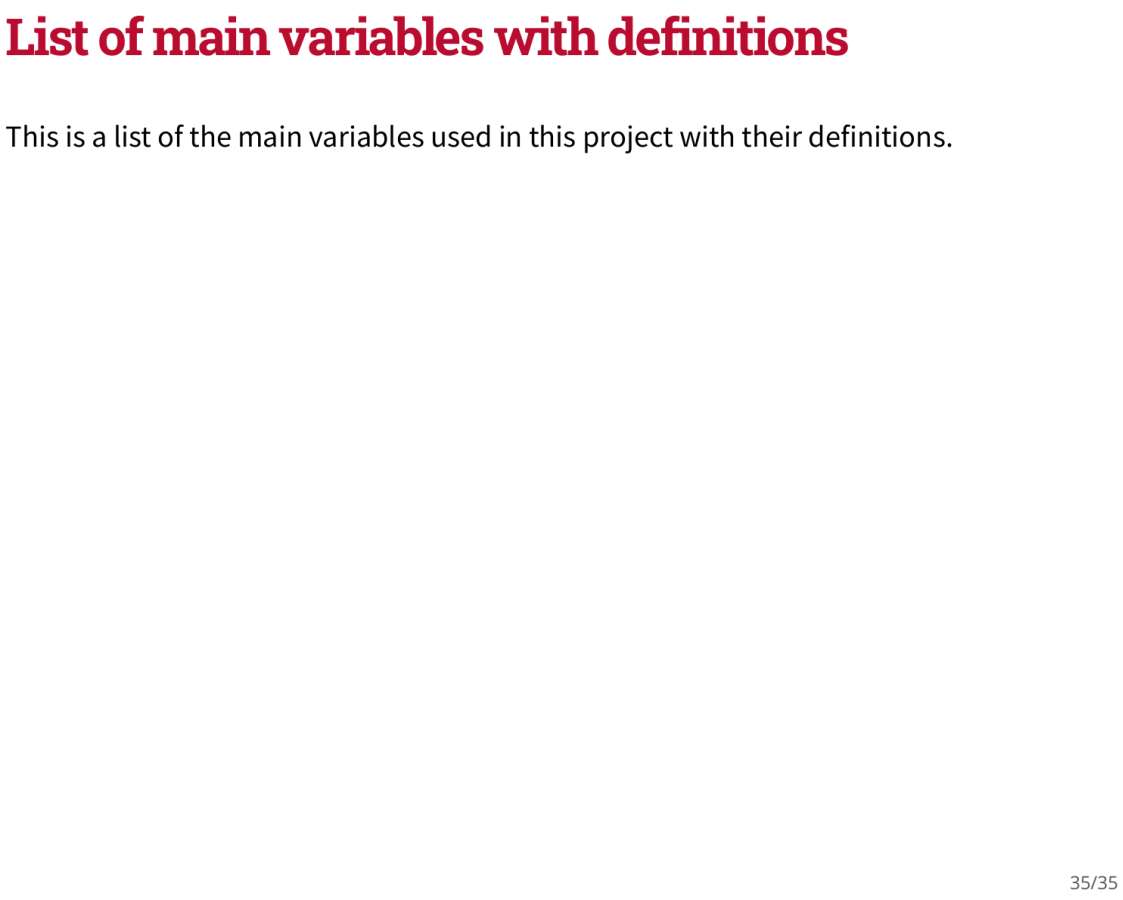

Figure 5.43: Slide 35: List of main variables with definitions

5.7 If your output doesn't match the reference

Most formatting mismatches come from a handful of recurring mistakes. See the Common Errors chapter for the diagnosis and fix for each — including:

- CSS in the wrong folder
- Wrong orientation (portrait instead of landscape)
- Knit-to-PDF instead of HTML → PDF
- YAML edits that break the rendering
- Background graphics not enabled in the print dialog

If you've checked all of those and still see a difference, email the teaching team. Attach **both** your `.Rmd` file and the `.pdf` you generated (the one with the wrong output), and include the print dialog settings you used.

6Common Errors

If something has gone wrong, look here first. This chapter is a diagnostic catalog of the failure modes we see most often, organized roughly in the order they tend to bite during the project workflow.

6.1 How to use this chapter

- **If you've gotten an error message** when running a script or knitting, search the relevant section below for the symptom.
- **If your output renders but looks wrong** when you compare it to the reference in the Submission chapter, jump to Output format errors — that's where the visual examples live.
- **If you've checked everything and still can't resolve it,** see Getting help at the end of this chapter.

Each entry follows the same pattern: **Symptom**, **Cause**, **Fix**.

6.2 Setup errors

6.2.1 CSS not loading

Symptom: Your knitted slides have no UGA styling — plain text, no colors, default fonts.

Cause: Either (a) `project_slidedeck.css` is anywhere other than `Project/css/`, or (b) the `.Rmd` file you're knitting isn't in your `Project/` root (e.g., you left it in your Downloads folder), so the relative path `css/project_slidedeck.css` in the Rmd's YAML can't find the CSS.

Fix: Confirm both files are in the right places: the CSS in `Project/css/` AND the `.Rmd` in your `Project/` root. Open RStudio by double-clicking `Project.Rproj`, then re-knit. Visual example of what the bad output looks like: Wrong CSS.

6.2.2 Working from your Downloads folder

Symptom: "File not found" errors when running scripts or knitting. R can't see your data files even though they exist.

Cause: RStudio's working directory is your Downloads folder, not your `Project/` folder. The `here()` paths in scripts and the Rmd resolve to the wrong place.

Fix: Move all files into `Project/` and always open RStudio by double-clicking `Project.Rproj`. Never work from Downloads.

6.2.3 Required files missing or in the wrong subfolder

Symptom: "Cannot open the connection" or "object not found" errors during a script run or knit.

Cause: A required file isn't where the script or Rmd expects it (e.g., `LF.R` is in your project root instead of `r/`, or `pppub24.csv` is missing from `data/`).

Fix: Run `check_setup_preproject.R` (pre-project) or `check_setup_project.R` (main project). The script tells you exactly which file is misplaced and where it should go.

6.3 Workflow errors

6.3.1 Knitting before running the R script

Symptom: Knit errors out with a "file not found" message pointing at `out/LF.csv` or `data/cpsmar_e.csv`.

Cause: You haven't run `LF.R` (pre-project) or `cpsmar_e.R` (main project) yet, so the file the Rmd is trying to read doesn't exist.

Fix: Run the relevant R script first (open it in the editor, Select All, click Run), then knit the Rmd.

6.3.2 Variable name confusion

Symptom: Knit errors out mid-document with "object not found" — a chunk references `cpsmar_a` but you defined `cpsmar_e` (or similar).

Cause: Typo or misalignment between chunks. You defined the analysis sample with one name, then referenced a different name in a downstream chunk.

Fix: Read each chunk carefully. Match data-frame names exactly across chunks. The variable structure in the template is: `cpsmar_e` (the extract) → `cpsmar_a` (the analysis sample) → `singles` (the singles subset).

6.3.3 Touching the setup chunk

Symptom: Output looks broken or knit fails in unexpected ways (missing libraries, scientific notation issues, formatting wrong).

Cause: You changed `include = FALSE` to `include = TRUE` on the setup chunk, or otherwise modified the setup chunk's contents.

Fix: Restore the setup chunk to its original state. The `eval = FALSE → TRUE` toggle only applies to content code chunks (`btl1`, `btl2`, `mi1`, etc.), never the setup chunk.

6.4 Rendering errors (the knit step)

6.4.1 Leaving `eval = FALSE` on a completed chunk

Symptom: Your code is correct but the output doesn't appear in the knitted slide. The chunk shows the code but no result below it.

Cause: The chunk option `eval = FALSE` is still set. R Markdown displays the code but skips running it during knit.

Fix: Change `eval = FALSE` to `eval = TRUE` on every content chunk you've completed. Re-knit.

6.4.2 Editing the YAML beyond the author line

Symptom: The knit produces unexpected output — different layout, missing CSS, wrong document class, no slide breaks.

Cause: You changed something in the YAML besides the `author:` line (e.g., the `output:` value, the `css:` reference, the `widescreen:` setting).

Fix:

1. Identify which Rmd is broken: `pre_project.Rmd` or `project.Rmd`.
2. Copy the canonical YAML block from the relevant chapter:
   - **Pre-project:** the YAML block is in Pre-project Step 4.
   - **Main project:** the YAML block is in Main Project — The YAML block.
3. Open your working Rmd. Replace your current YAML with the YAML you just copied.
4. Re-add your name on the `author:` line.
5. Re-knit.

6.4.3 Using the Knit dropdown instead of the Knit button

Symptom: Output is a Word document, a LaTeX-styled PDF, or some format that doesn't match the slide deck reference.

Cause: You clicked the small dropdown arrow next to the Knit button and selected "Knit to PDF / Word / etc." That overrides the output format specified in the Rmd's YAML.

Fix: Click the **Knit** button itself, not the dropdown arrow. The YAML already tells R Markdown to knit to HTML; just trust the button.

6.5 Output format errors

These are the canonical "submission rejected" failure modes, each shown with a real example from a prior submission. If your output matches any of these images, your submission will receive a 0.

6.5.1 Wrong CSS

Symptom: Output is in landscape but has no UGA styling — plain text, gray gradients, default browser fonts.

Cause: `project_slidedeck.css` was not found at knit time (usually because it's not in `Project/css/`).

Fix: Verify the CSS file is in `Project/css/`. Verify the Rmd you are working on is in `Project/`, not somewhere else (like `Downloads/`). Re-knit.

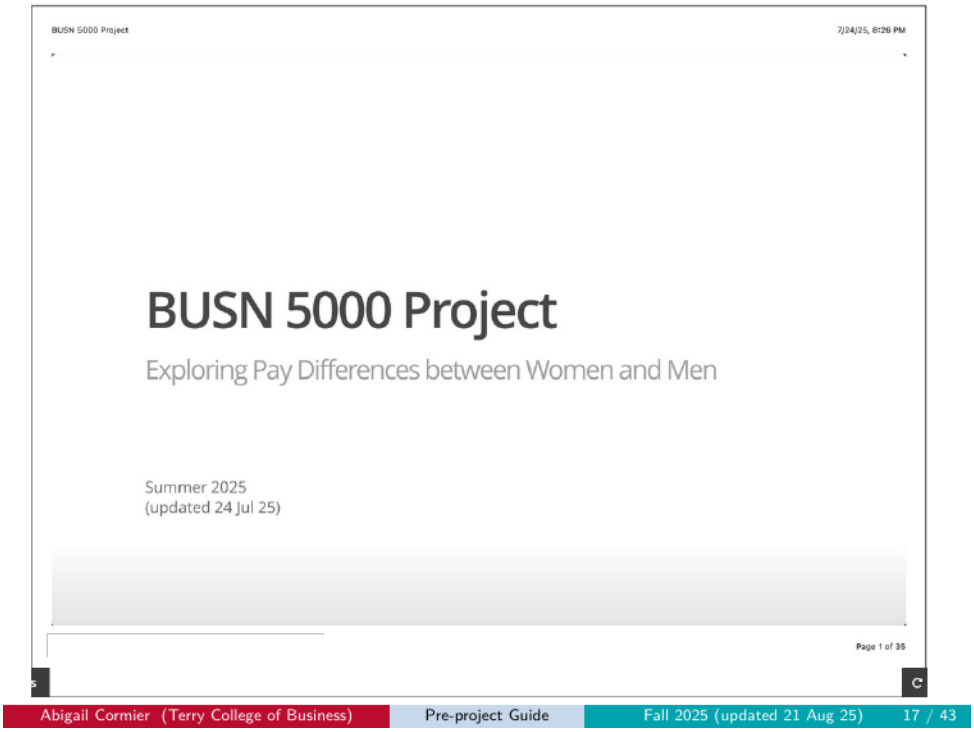

Figure 6.1: Wrong CSS example

6.5.2 Wrong CSS + wrong orientation

Symptom: Output is portrait AND has no UGA styling.

Cause: Both CSS is missing and the print dialog saved as portrait.

Fix: Move CSS to `Project/css/`. Verify the Rmd you are working on is in `Project/`, not somewhere else (like `Downloads/`). Re-knit, then save with **Landscape** orientation.

Figure 6.2: Wrong CSS, wrong orientation example

6.5.3 Wrong orientation

Symptom: Correct UGA styling, but the PDF is portrait.

Cause: Print dialog defaulted to portrait and you didn't change it.

Fix: In the browser's print dialog, set **Layout / Orientation to Landscape** before saving.

Figure 6.3: Wrong orientation example

6.5.4 Long landscape

Symptom: Landscape PDF, but the slide content is stretched horizontally across a single very wide page instead of broken into separate slide pages.

Cause: A print dialog scaling setting fit multiple slides onto one wide page.

Fix: Reset print dialog scaling to default. Make sure **Pages per sheet = 1**.

Figure 6.4: Long landscape example

6.5.5 Portrait

Symptom: Output is portrait. (Same as "Wrong orientation" above, isolated here as an example.)

Cause: Print dialog defaulted to portrait.

Fix: Set Layout to **Landscape**.

Figure 6.5: Portrait example

6.5.6 Portrait + wrong CSS

Symptom: Output is portrait AND has no UGA styling. Same underlying causes as the "Wrong CSS + wrong orientation" case, isolated here as a distinct example pattern.

Fix: Move CSS to `Project/css/`. Verify the Rmd you are working on is in `Project/`, not somewhere else (like `Downloads/`). Re-knit, then save with **Landscape** orientation.

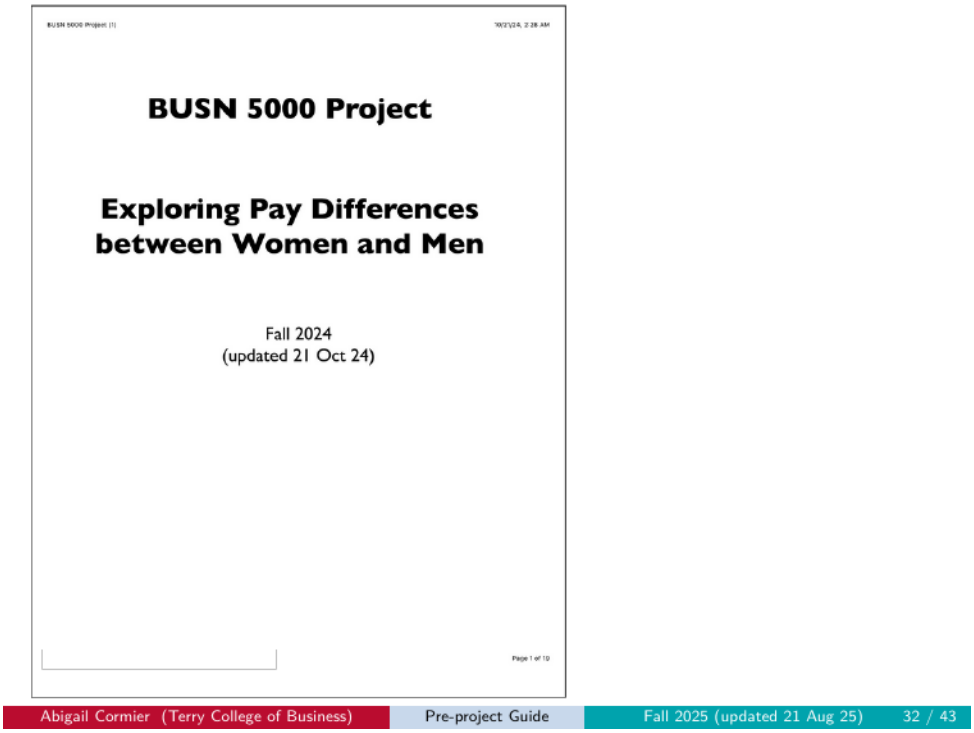

Figure 6.6: Portrait + wrong CSS example

6.5.7 LaTeX slides (knit-to-PDF, slide format)

Symptom: Output is a slide deck rendered with LaTeX/Beamer styling — different fonts, different layout, no UGA brand colors.

Cause: You used the Knit dropdown to select "Knit to PDF" with slide output, overriding the Rmd's HTML-based ioslides format.

Fix: Use the Knit button (not the dropdown). The Rmd's YAML specifies `ioslides_presentation` for a reason. If your YAML has also been modified (which often goes hand-in-hand with this error), restore it by following the 6-step process under Editing the YAML beyond the author line. Then re-knit.

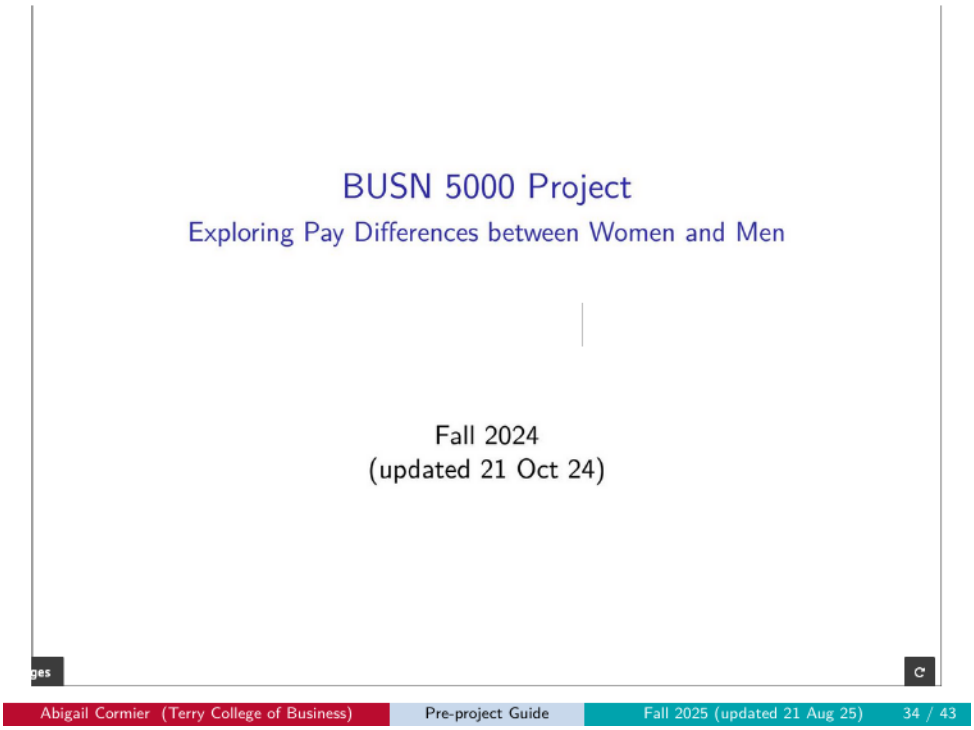

Figure 6.7: LaTeX slides example

6.5.8 LaTeX document (knit-to-PDF, document format)

Symptom: Output is a regular PDF document, not a slide deck at all. Looks like a paper or article.

Cause: You used the Knit dropdown to select "Knit to PDF" with document output, AND likely modified the YAML to change the output format.

Fix: Restore the YAML. Knit with the button. Save as PDF from the HTML output.

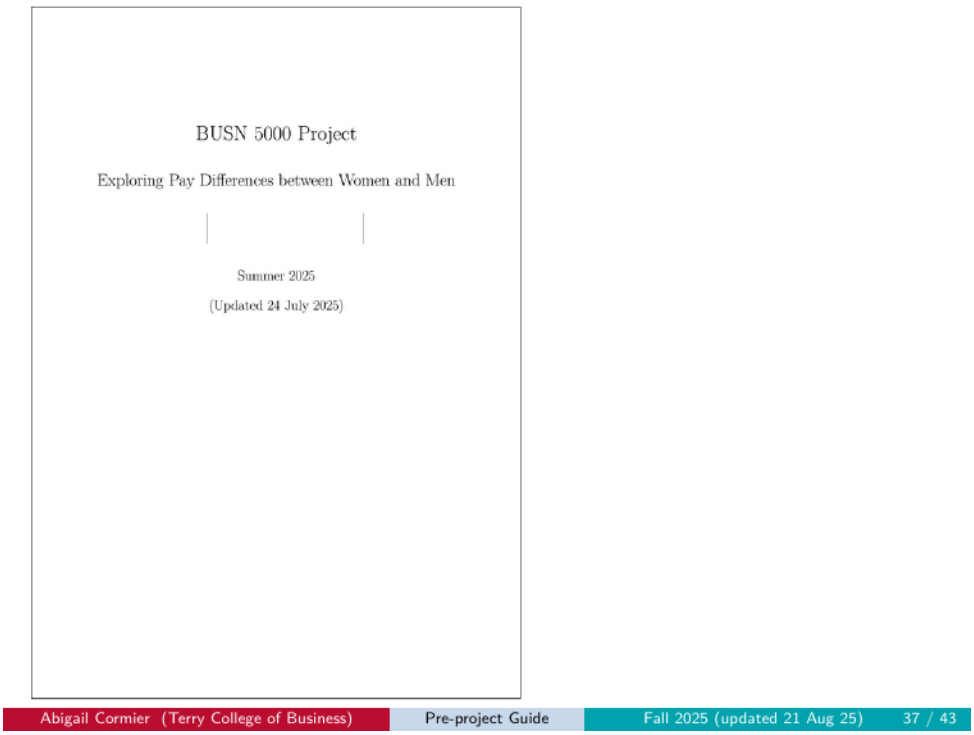

Figure 6.8: LaTeX document example

6.5.9 Not knit

Symptom: Submission is the raw `.Rmd` file, or a file that was never actually knit — no rendering at all.

Cause: You uploaded the wrong file (the `.Rmd` source instead of the saved-as-PDF output).

Fix: Knit the Rmd to HTML, save the HTML as PDF using your browser's print dialog, then upload that PDF.

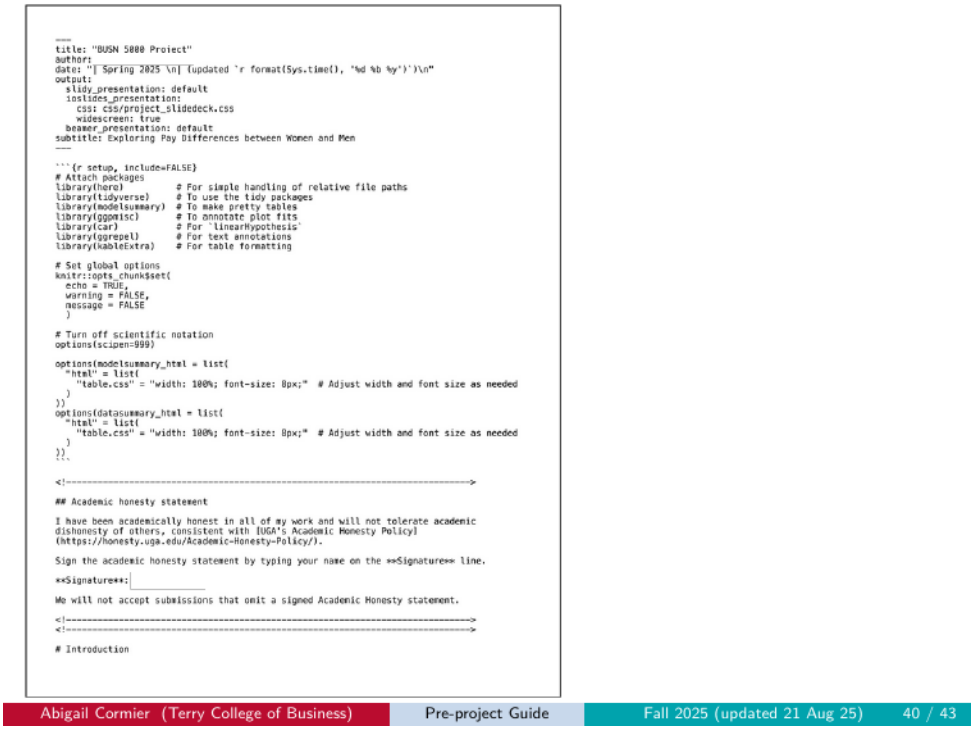

Figure 6.9: Not knit example

6.5.10 Wrong file

Symptom: Submission is something other than the intended project artifact — an old version, a template, an unrelated document.

Cause: You uploaded the wrong file.

Fix: Locate the correct PDF in your `Project/` folder and re-upload. Double-check the filename matches the convention.

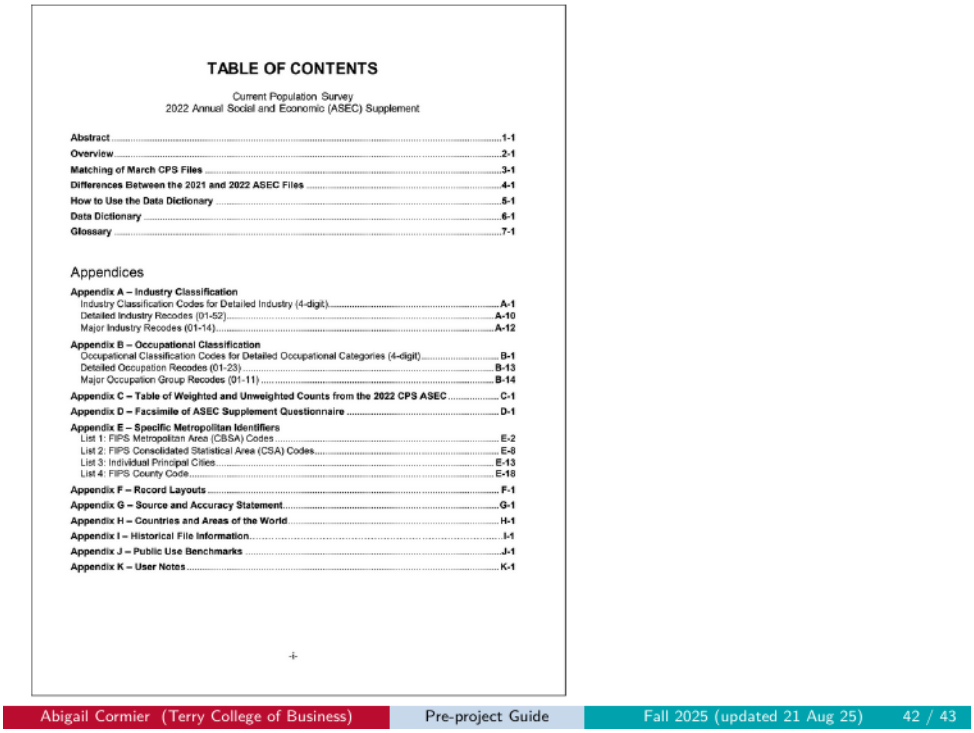
TABLE OF CONTENTS

Current Population Survey
2022 Annual Social and Economic (ASEC) Supplement

Abstract 1-1
Overview 2-1
Matching of March CPS Files 3-1
Differences Between the 2021 and 2022 ASEC Files 4-1
How to Use the Data Dictionary 5-1
Data Dictionary 6-1
Glossary 7-1

Appendices

Appendix A – Industry Classification
Industry Classification Codes for Detailed Industry (4-digit) A-1
Detailed Industry Recodes (01-52) A-10
Major Industry Recodes (01-14) A-12
Appendix B – Occupational Classification
Occupational Classification Codes for Detailed Occupational Categories (4-digit) B-1
Detailed Occupation Recodes (01-23) B-13
Major Occupation Group Recodes (01-11) B-14
Appendix C – Table of Weighted and Unweighted Counts from the 2022 CPS ASEC C-1
Appendix D – Facsimile of ASEC Supplement Questionnaire D-1
Appendix E – Specific Metropolitan Identifiers
List 1: FIPS Metropolitan Area (CBSA) Codes E-2
List 2: FIPS Consolidated Statistical Area (CSA) Codes E-8
List 3: Individual Principal Cities E-13
List 4: FIPS County Code E-18
Appendix F – Record Layouts F-1
Appendix G – Source and Accuracy Statement G-1
Appendix H – Countries and Areas of the World H-1
Appendix I – Historical File Information I-1
Appendix J – Public Use Benchmarks J-1
Appendix K – User Notes K-1

Abigail Cormier (Terry College of Business) Pre-project Guide Fall 2025 (updated 21 Aug 25) 42 / 43

Figure 6.10: Wrong file example

6.6 Content errors

6.6.1 Numbers in your paragraph don't match `LF.csv` (pre-project)

Symptom: Your slide 3 paragraph cites numbers that don't reflect your own data — they came from a classmate or from a previous semester.

Cause: You copied numbers from a peer instead of running `LF.R` yourself, or `LF.R` didn't actually run and the values you wrote down were stale.

Fix: Run `LF.R` yourself. Open `out/LF.csv` (or print `lf` in the console). Fill in your slide 3 paragraph with those values, rounded to two decimal places.

6.6.2 Wrong filename

Symptom: Your file was uploaded but appears under an unexpected name in Gradescope, or you can't tell which submission is which in your `Project/` folder later.

Cause: The filename didn't match the required pattern.

Fix: Rename your file to the correct convention (`firstname_lastname_pre.pdf` / `firstname_lastname_progress` / `firstname_lastname_project.pdf`) and re-upload.

6.7 Getting help

If your error isn't in this chapter, or you've checked and still can't resolve it, here's how to ask for help.

! Communication policy (summarized from the syllabus)

- **For content-related questions**, contact **TAL staff** first. If TAL staff can't solve the problem, then email Abbi or Dr. Cornwell.
- **For administrative questions**, email **Abbi first**. She'll loop in Dr. Cornwell if needed.
- **One thread per issue.** Don't send the same problem to both Abbi and Dr. Cornwell on separate emails. We will loop the other in if needed. Duplicate emails waste resources for your classmates.
- **No content review by email before the deadline.** We will not pre-grade or review your project ahead of the due date.
- **Email format requirements:**
  - **Subject line:** include your section time and a brief category (e.g., "coding error" or "homework question").
  - **Greeting:** "Hey" is not a proper greeting. "Dear Abbi" or "Dear Ms. Cormier" is.
  - **Body:** a clear description of the problem and what you've tried. For coding issues, paste your code in the email body. For project errors, attach your `.Rmd` file and paste the error message in the body. **Do not send screenshots or photos.**
  - **Closing:** a proper closing (e.g., "Respectfully, your_name").

Omission of any of these features may cause your message to be rejected.

The full communication policy lives in the course syllabus on eLC — see the **Communication** section. If your question is about email mechanics rather than course content, check there first.

7R and AI Resources

A curated list of resources for learning and using R, and for working with AI as part of your data-science practice. Nothing here is required reading — but each item below has earned its place. Skim it now, bookmark it, and return as you go.

7.1 Resources for learning and using R

7.1.1 IDEs for R

RStudio remains the most widely used IDE for R, and, like R, it is open-source. Its "Help" tab is a gateway to a wide range of R and RStudio resources. However, Posit (the company behind RStudio) has released Positron, a next-generation IDE built for data science with both R and Python. Positron combines the familiar feel of RStudio with the extensibility of VS Code, and includes built-in AI assistance via Positron Assistant. If you're comfortable with RStudio, stick with it; if you're curious about a more modern interface or plan to work across R and Python, Positron is worth exploring.

7.1.2 Learning R

Norm Matloff, a computer scientist and statistician at UC-Davis, has a nice, fast introduction to (base) R. He also has interesting takes on base R vs the Tidyverse and the R vs Python debate in data science. For a more in-depth treatment, consult the widely used R for Data Science by Wickham and Grolemund. They will guide you through the Tidyverse. Tidy Data Tutor helps you visualize your data analysis pipelines. James Scott has a great introduction to R that more clearly maps onto BUSN 5000.

You will find a list of econometrics packages for R at the Comprehensive R Archive Network (CRAN). You might prefer the causal inference task view or machine learning task view.

Learning R Markdown or Quarto should go hand-in-hand with learning R. For R Markdown, get started here and keep a cheatsheet handy. You can get started with Quarto here.

7.1.3 Workflow

At least as important as learning R is understanding basic workflow principles. R-bloggers has a beginner's guide using RStudio Projects. You can find some of the same principles in chapter 2 of Kieran Healy's Data Visualization book (also linked on the course schedule). Matt Gentzkow and Jesse Shapiro provide a fuller take on workflow principles from the perspective of academic social scientists (also posted on eLC). They even provide a manual for doing as they do.

For a perspective specifically on coding practices in data science education, Pruim, Gîrjău, and Horton (2023) make the case in "Fostering Better Coding Practices for Data Scientists" (*Harvard Data Science Review*) that good coding habits matter even for beginners. Their argument: it's far easier to learn good practices alongside R than to unlearn bad habits later. They organize their guidance into a practical top-10 list. Worth reading early in your data-science career.

Suffice it to say that Healy's book is a great introduction to the wonders of ggplot, which is not R, but indispensable to analysis done in R.

7.2 AI resources

7.2.1 Working with AI

My take is that you should view AI chatbots, like ChatGPT, Claude, and Gemini, as exceedingly productive collaborators with whom you should learn to work, much like you would with a human. Unfortunately, there are no secret prompting strategies or special step-by-step manuals (also like with a human). For some practical guidance along these lines, I recommend that you follow Ethan Mollick's Substack. If you want to swim in the deeper end of the pool, check out the Substack, Don't Worry About the Vase.

7.2.2 AI for coding

One place gen AI really helps is in coding. You can get coding help from chat interfaces, but you may prefer a more integrated experience. Several options exist:

- Positron Assistant: If you use Positron, this built-in AI assistant understands your R environment, loaded data, and console history. It integrates GitHub Copilot for code completions and Claude for chat and agent mode.
- GitHub Copilot: Works in RStudio, VS Code, and other IDEs. Here is how to get free access as a student. Here are instructions for RStudio integration.
- Claude Code: A command-line tool that can read and modify files in your project directory, run commands, and understand your codebase context. Simon Couch has written useful posts on using Claude Code for R package development.
- Cursor: An AI-first code editor built on VS Code.

Alternatively, you may find you can get by just vibe coding with a chatbot — describing what you want and iterating on the results.

7.2.3 Learning more about AI

To learn more about AI, including a comparison of different models, check out Generative AI for Beginners, a free course offered by Microsoft Cloud Advocates. OpenAI and Anthropic have their own AI academies when you want to step it up.